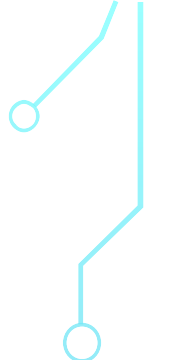
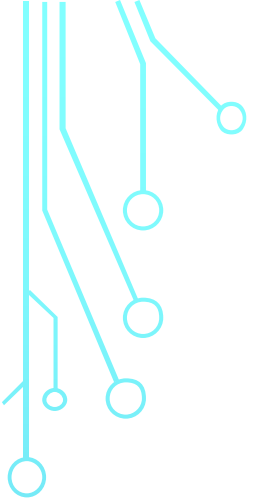


A decorative graphic on the left side of the slide, consisting of a network of thin, gold-colored lines and small circles, resembling a circuit board or a stylized tree structure.

Function Stack Frames

COMP211 Connor McMahon



Reminder

Make sure you have a TA verify your setup from HW0 by Thursday.

Stack Frames

- The **Stack**:
 - An area of memory used to manage function calls
- **Stack Frame**:
 - Each stack frame is like a mini workspace for a function.
 - It holds everything the function needs to do its job, including:
 - **Return Address**: The location in the program to return to after the function finishes.
 - **Arguments**: Values passed into the function.
 - **Local Variables**: Variables declared inside the function.

Motivation

- Understanding stack frames helps you understand what is actually happening when functions call other functions.
- It also explains several important programming concepts, including:
 - Why local variables disappear when a function returns
 - How nested and recursive function calls work
 - Why returning a pointer to a local variable can cause problems
 - How the program knows where to continue after a function returns
 - Why very deep recursion can cause a **stack overflow**
 - How debuggers display the **call stack** when a program crashes

Caller and Callee

- **Caller**: the part of the program that calls a function
- **Callee**: the function that is called
- How a Function Call Works
 - The **caller** passes data (arguments) to the function
 - The **callee**:
 - Receives the arguments
 - Executes its code
 - Produces a result (return value)
 - Control returns to the caller

Stack Pointer

- **Stack pointer**

- Holds the memory address of the current top of the stack.
- When a function is called, the stack pointer is adjusted to make room for the new stack frame.
- When the function returns, it is adjusted back, effectively removing that frame.

Example 1

```
void C(int z) {  
    int c = z + 1;  
}  
  
void B(int y) {  
    int b = y * 2;  
    C(b);  
}  
  
void A() {  
    int a = 10;  
    B(a);  
}  
  
int main() {  
    A();  
}
```

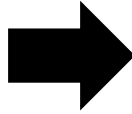
```
void C(int z) {  
    int c = z + 1;  
}  
  
void B(int y) {  
    int b = y * 2;  
    C(b);  
}  
  
void A() {  
    int a = 10;  
    B(a);  
}  
  
int main() {  
    A();  
}
```

Stack Frame:

- Each stack frame is like a mini workspace for a function.
- It holds everything the function needs to do its job, including:
 - **Return Address:** The location in the program to return to after the function finishes.
 - **Arguments:** Values passed into the function.
 - **Local Variables:** Variables declared inside the function.

```
void C(int z) {  
    int c = z + 1;  
}  
  
void B(int y) {  
    int b = y * 2;  
    C(b);  
}  
  
void A() {  
    int a = 10;  
    B(a);  
}  
  
int main() {  
    A();  
}
```

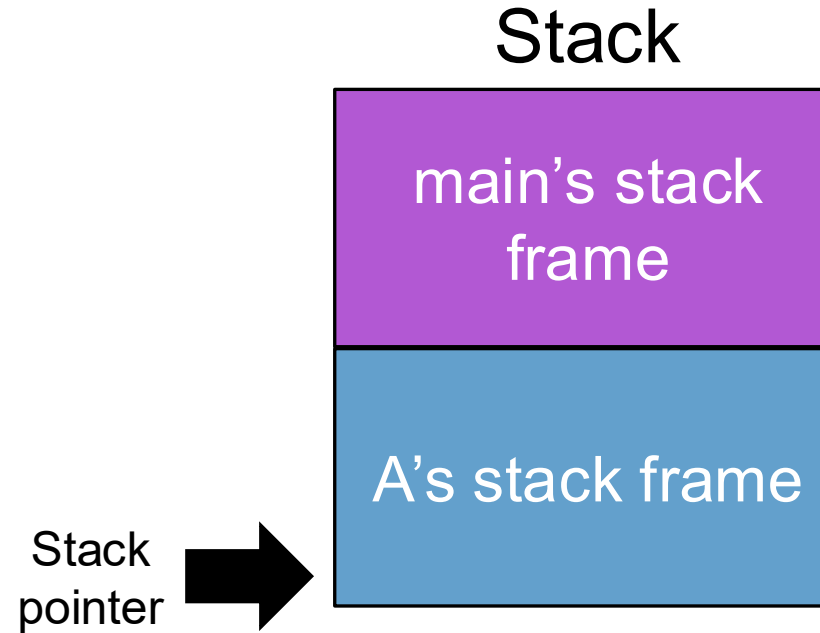
Stack
pointer



main's stack
frame

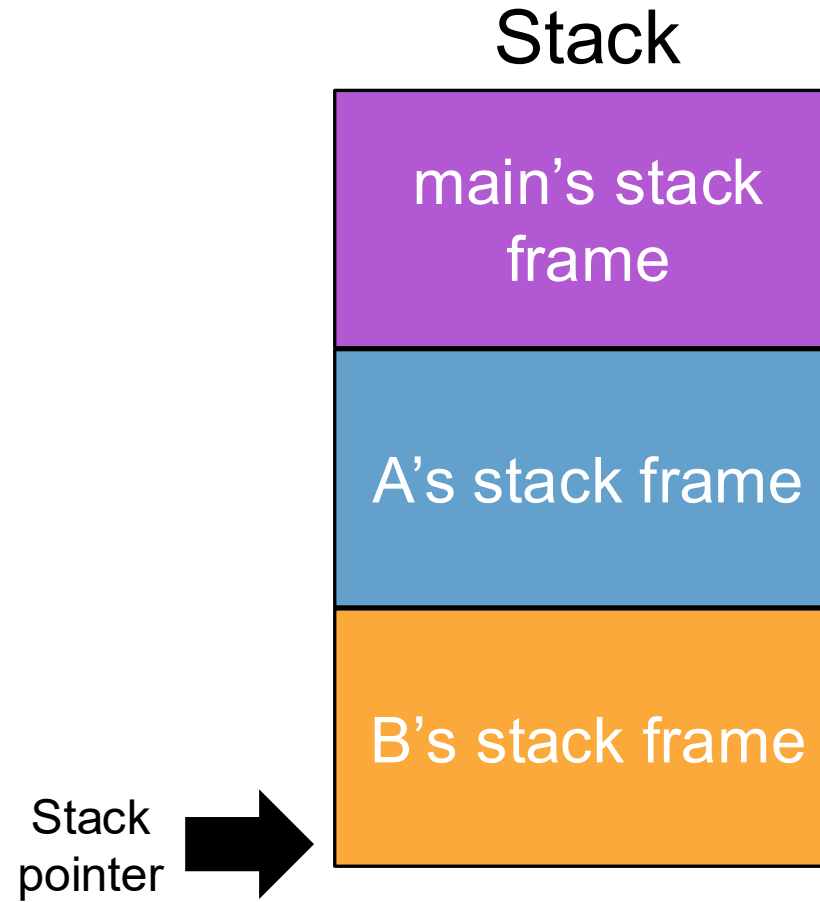
When main is
called, a new
stack frame is
created for it on
the stack.

```
void C(int z) {  
    int c = z + 1;  
}  
  
void B(int y) {  
    int b = y * 2;  
    C(b);  
}  
  
void A() {  
    int a = 10;  
    B(a);  
}  
  
int main() {  
    A();  
}
```



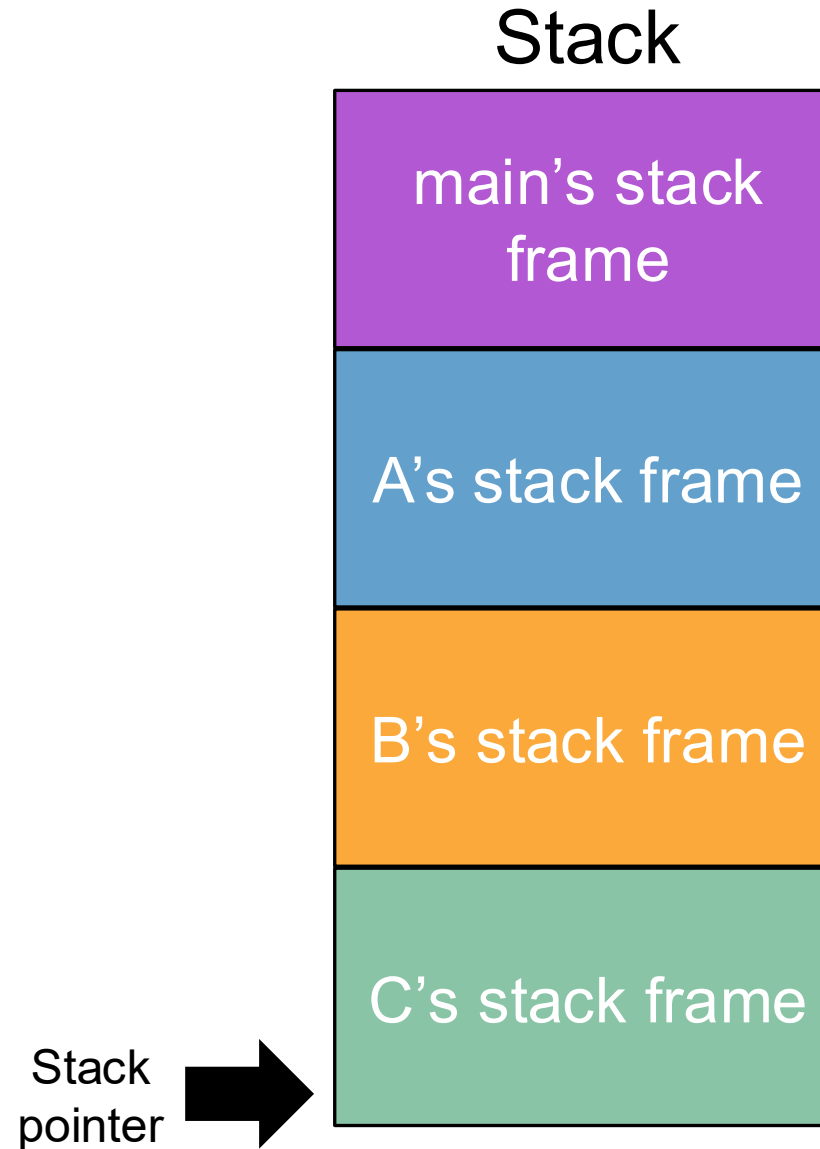
When main calls A, a new stack frame for A is created below main's frame, causing the stack to grow downward.

```
void C(int z) {  
    int c = z + 1;  
}  
  
void B(int y) {  
    int b = y * 2;  
    C(b);  
}  
  
void A() {  
    int a = 10;  
    B(a);  
}  
  
int main() {  
    A();  
}
```



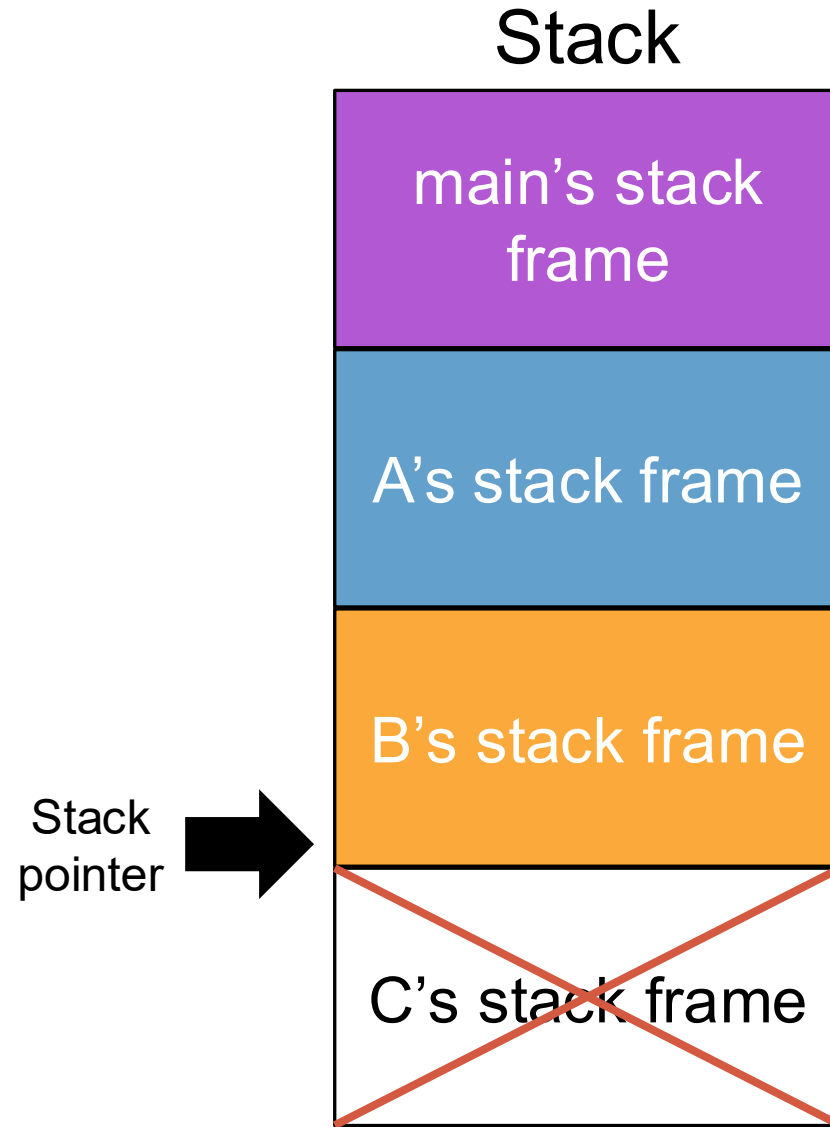
When A calls B, a new stack frame for B is created below A's frame, causing the stack to grow downward.

```
void C(int z) {  
    int c = z + 1;  
}  
  
void B(int y) {  
    int b = y * 2;  
    C(b);  
}  
  
void A() {  
    int a = 10;  
    B(a);  
}  
  
int main() {  
    A();  
}
```



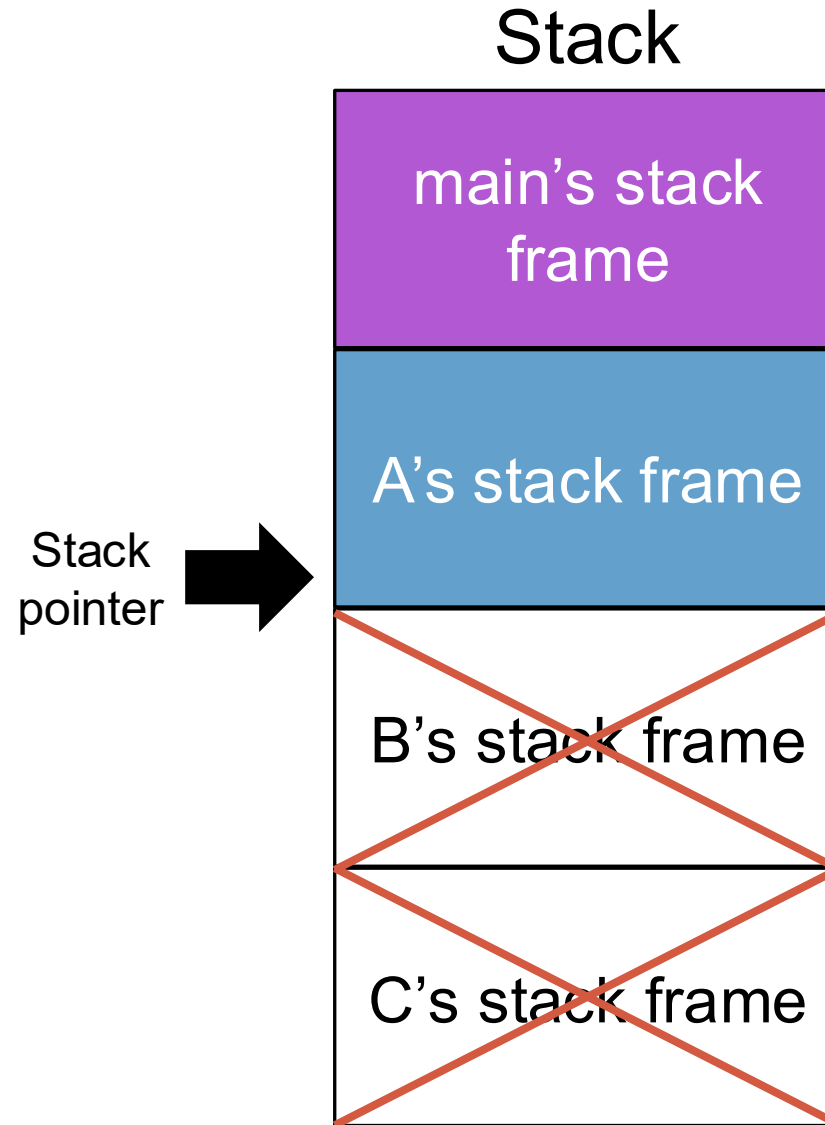
When B calls C,
a new stack
frame for C is
created below
B's frame,
causing the stack
to grow
downward.

```
void C(int z) {  
    int c = z + 1;  
}  
  
void B(int y) {  
    int b = y * 2;  
    C(b);  
}  
  
void A() {  
    int a = 10;  
    B(a);  
}  
  
int main() {  
    A();  
}
```



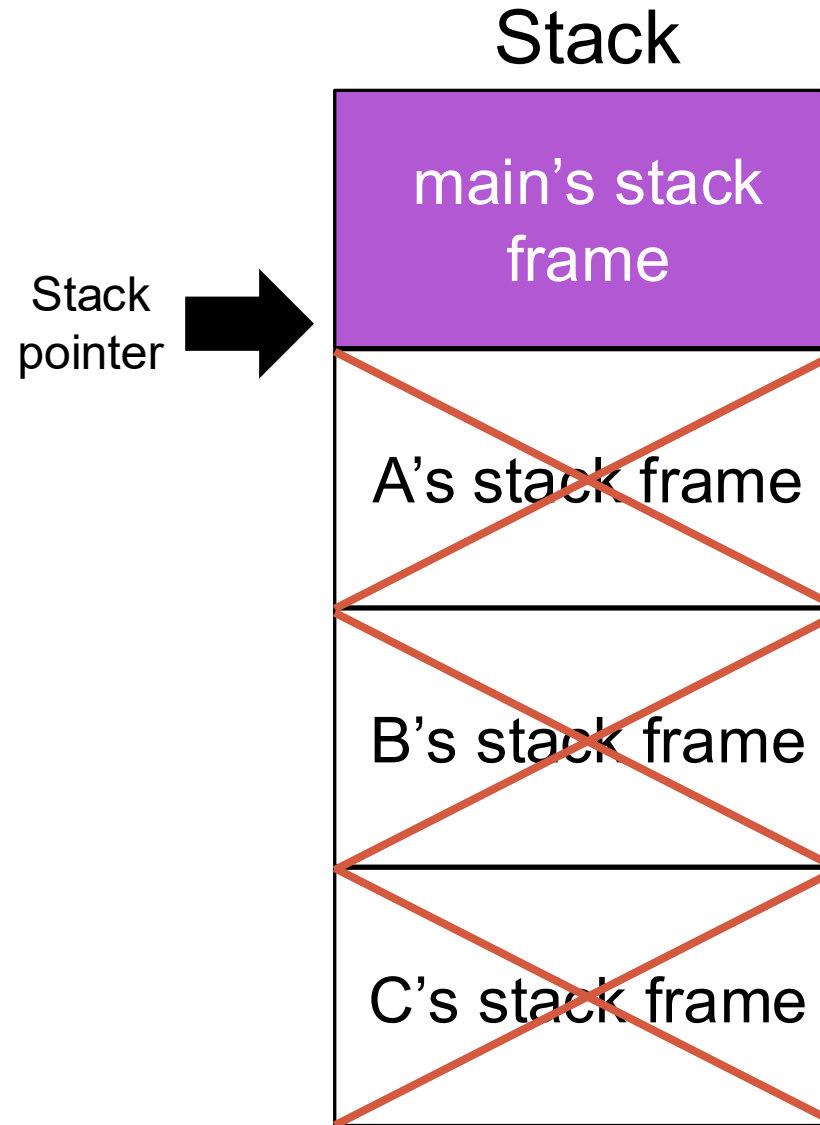
When C returns, the stack pointer moves back, effectively removing C's stack frame. The data from C may still remain in memory, but that memory is no longer valid and can be reused.

```
void C(int z) {  
    int c = z + 1;  
}  
  
void B(int y) {  
    int b = y * 2;  
    C(b);  
}  
  
void A() {  
    int a = 10;  
    B(a);  
}  
  
int main() {  
    A();  
}
```



When B returns, the stack pointer moves back, effectively removing B's stack frame. The data from B may still remain in memory, but that memory is no longer valid and can be reused.

```
void C(int z) {  
    int c = z + 1;  
}  
  
void B(int y) {  
    int b = y * 2;  
    C(b);  
}  
  
void A() {  
    int a = 10;  
    B(a);  
}  
  
int main() {  
    A();  
}
```



When A returns, the stack pointer moves back, effectively removing A's stack frame. The data from A may still remain in memory, but that memory is no longer valid and can be reused.

Example 2

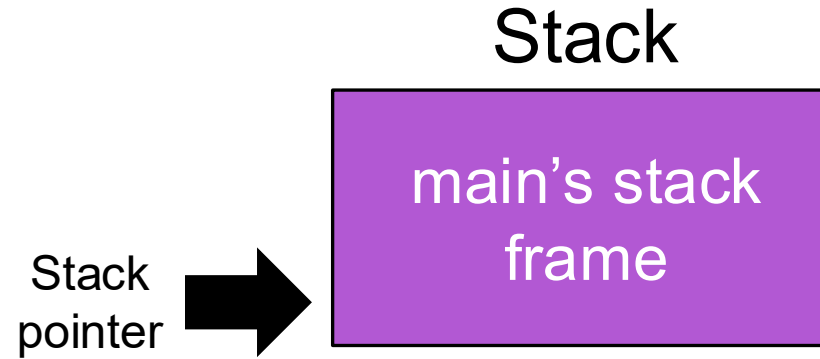
```
int square(int x) {  
    int result = x * x;  
    return result;  
}  
  
int addOne(int x) {  
    int result = x + 1;  
    return result;  
}  
  
int calculate() {  
    int answer =  
        addOne(square(5));  
    return answer;  
}  
  
int main() {  
    calculate();  
}
```

```
int square(int x) {  
    int result = x * x;  
    return result;  
}  
  
int addOne(int x) {  
    int result = x + 1;  
    return result;  
}  
  
int calculate() {  
    int answer =  
        addOne(square(5));  
    return answer;  
}  
  
int main() {  
    calculate();  
}
```

Stack Frame:

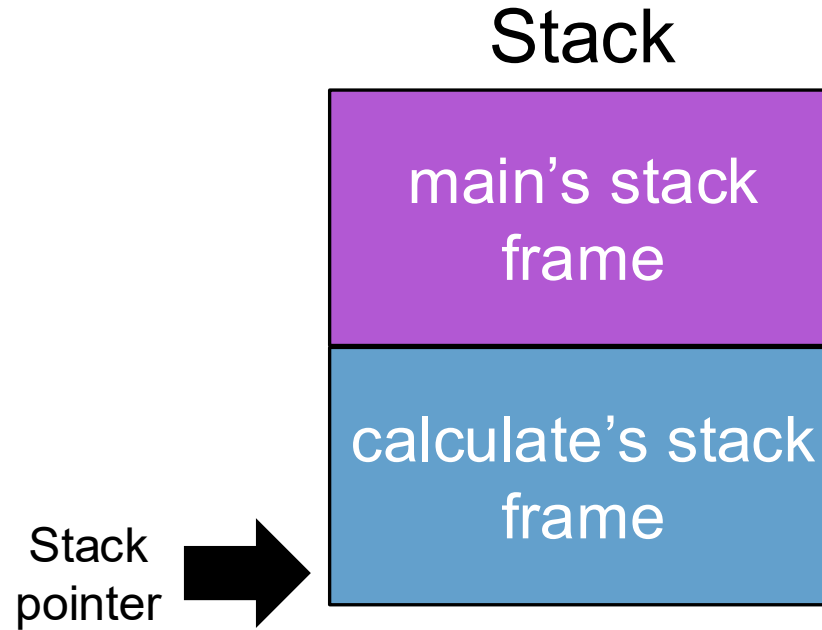
- Each stack frame is like a mini workspace for a function.
- It holds everything the function needs to do its job, including:
 - **Return Address:** The location in the program to return to after the function finishes.
 - **Arguments:** Values passed into the function.
 - **Local Variables:** Variables declared inside the function.

```
int square(int x) {  
    int result = x * x;  
    return result;  
}  
  
int addOne(int x) {  
    int result = x + 1;  
    return result;  
}  
  
int calculate() {  
    int answer =  
    addOne(square(5));  
    return answer;  
}  
  
int main() {  
    calculate();  
}
```



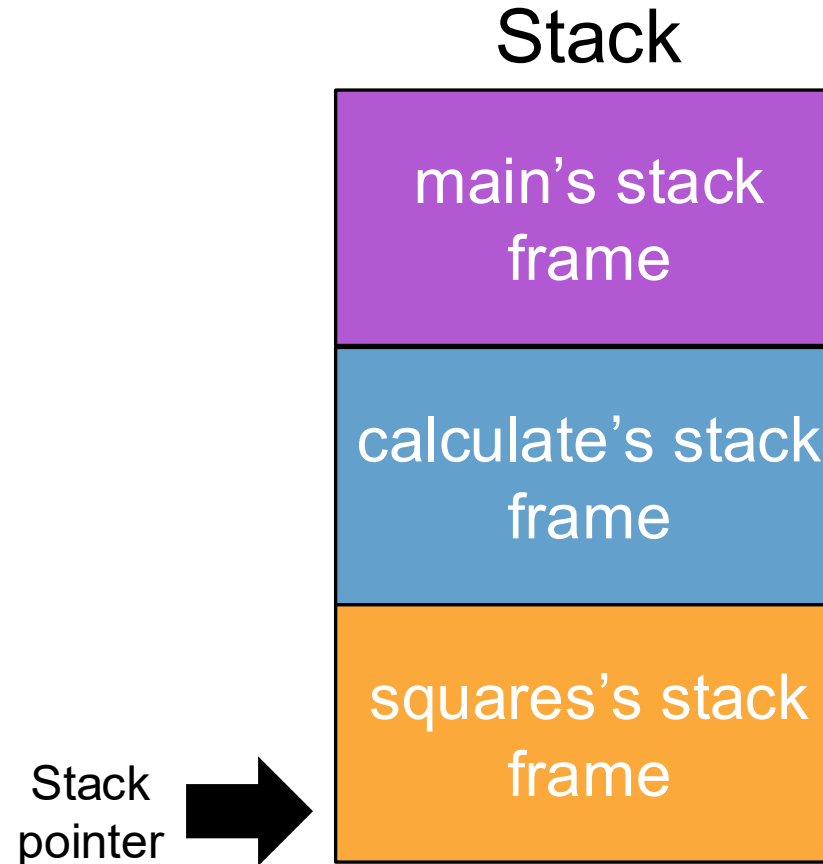
When main is called, a new stack frame is created for it on the stack.

```
int square(int x) {  
    int result = x * x;  
    return result;  
}  
  
int addOne(int x) {  
    int result = x + 1;  
    return result;  
}  
  
int calculate() {  
    int answer =  
        addOne(square(5));  
    return answer;  
}  
  
int main() {  
    calculate();  
}
```



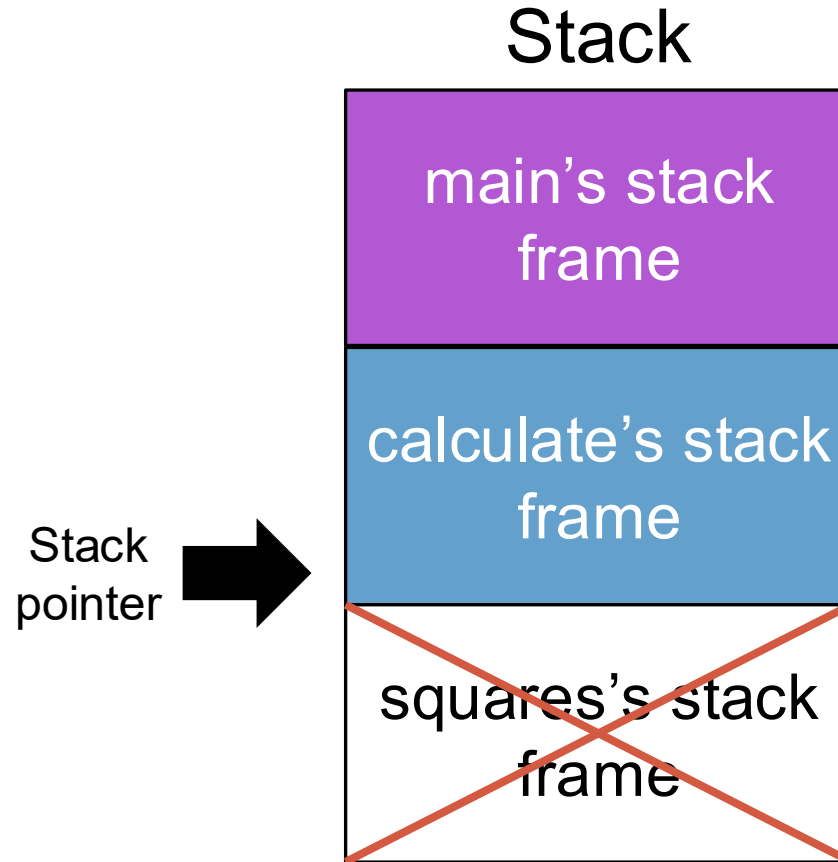
When main calls calculate, a new stack frame for calculate is created below main's frame, causing the stack to grow downward.

```
int square(int x) {  
    int result = x * x;  
    return result;  
}  
  
int addOne(int x) {  
    int result = x + 1;  
    return result;  
}  
  
int calculate() {  
    int answer =  
        addOne(square(5));  
    return answer;  
}  
  
int main() {  
    calculate();  
}
```



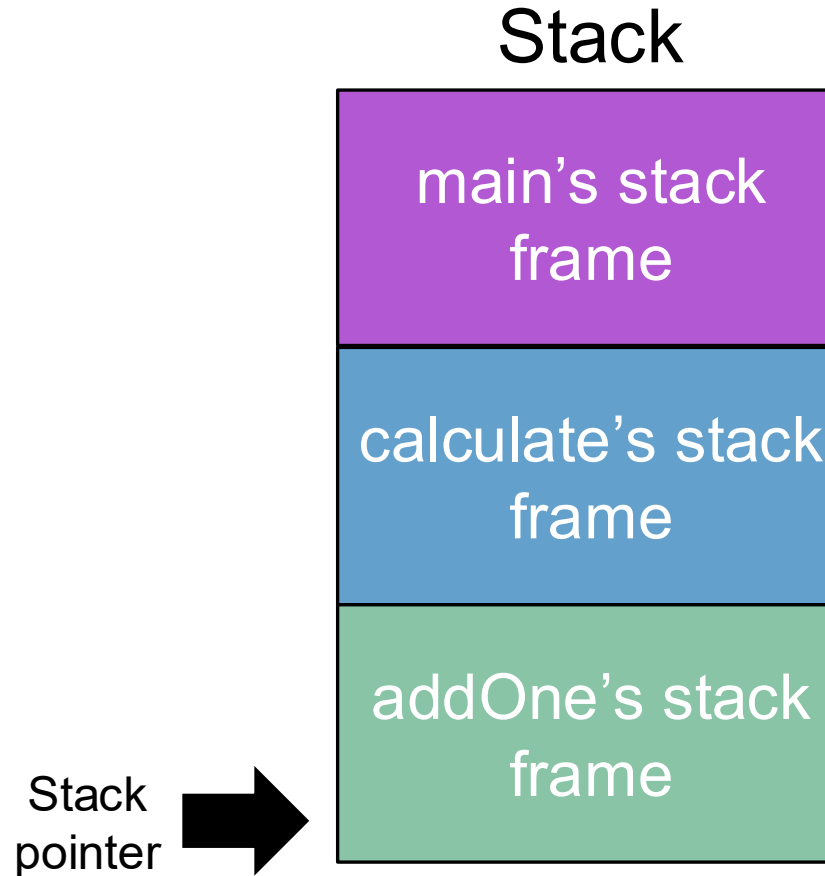
When calculate calls square, a new stack frame for square is created below calculate's frame, causing the stack to grow downward.

```
int square(int x) {  
    int result = x * x;  
    return result;  
}  
  
int addOne(int x) {  
    int result = x + 1;  
    return result;  
}  
  
int calculate() {  
    int answer =  
        addOne(square(5));  
    return answer;  
}  
  
int main() {  
    calculate();  
}
```



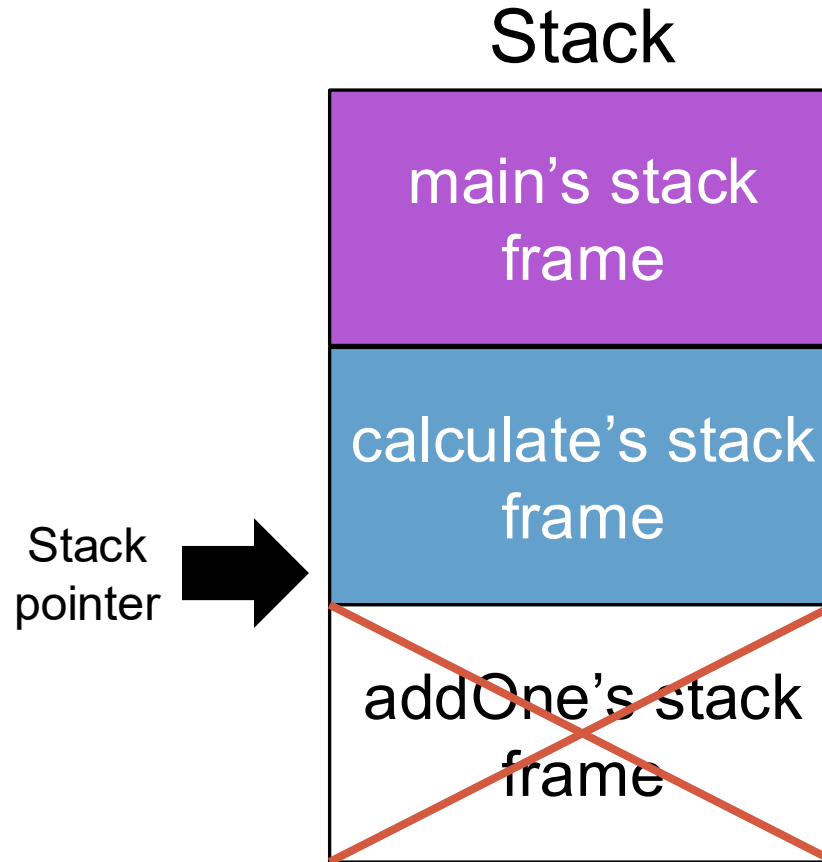
When square returns, the stack pointer moves back, effectively removing square's stack frame. The data from square may still remain in memory, but that memory is no longer valid and can be reused.

```
int square(int x) {  
    int result = x * x;  
    return result;  
}  
  
int addOne(int x) {  
    int result = x + 1;  
    return result;  
}  
  
int calculate() {  
    int answer =  
        addOne(square(5));  
    return answer;  
}  
  
int main() {  
    calculate();  
}
```



When calculate calls addOne, a new stack frame for addOne is created below calculate's frame, causing the stack to grow downward. This new frame may reuse the same stack memory that was previously occupied by square's frame.

```
int square(int x) {  
    int result = x * x;  
    return result;  
}  
  
int addOne(int x) {  
    int result = x + 1;  
    return result;  
}  
  
int calculate() {  
    int answer =  
        addOne(square(5));  
    return answer;  
}  
  
int main() {  
    calculate();  
}
```

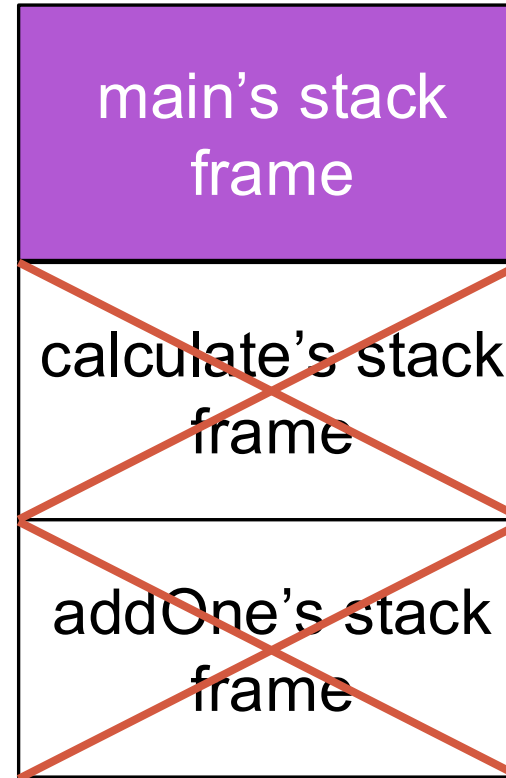


When `addOne` returns, the stack pointer moves back, effectively removing `addOne`'s stack frame. The data from `addOne` may still remain in memory, but that memory is no longer valid and can be reused.

```
int square(int x) {  
    int result = x * x;  
    return result;  
}  
  
int addOne(int x) {  
    int result = x + 1;  
    return result;  
}  
  
int calculate() {  
    int answer =  
        addOne(square(5));  
    return answer;  
}  
  
int main() {  
    calculate();  
}
```

Stack
pointer →

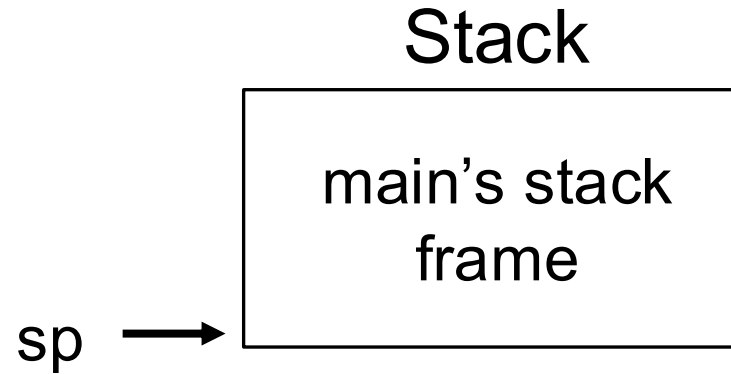
Stack



When calculate returns, the stack pointer moves back, effectively removing calculate's stack frame. The data from calculate may still remain in memory, but that memory is no longer valid and can be reused.

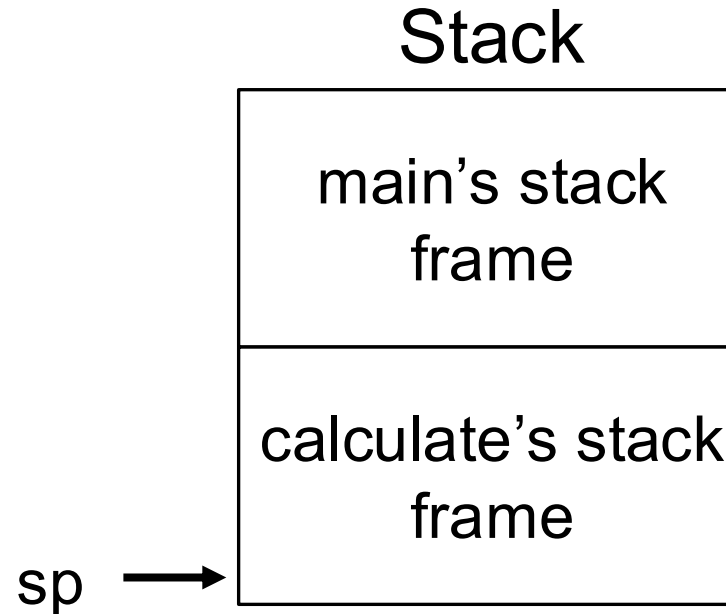
How you will draw this memory diagram on paper

```
int square(int x) {  
    int result = x * x;  
    return result;  
}  
  
int addOne(int x) {  
    int result = x + 1;  
    return result;  
}  
  
int calculate() {  
    int answer =  
        addOne(square(5));  
    return answer;  
}  
  
int main() {  
    calculate();  
}
```



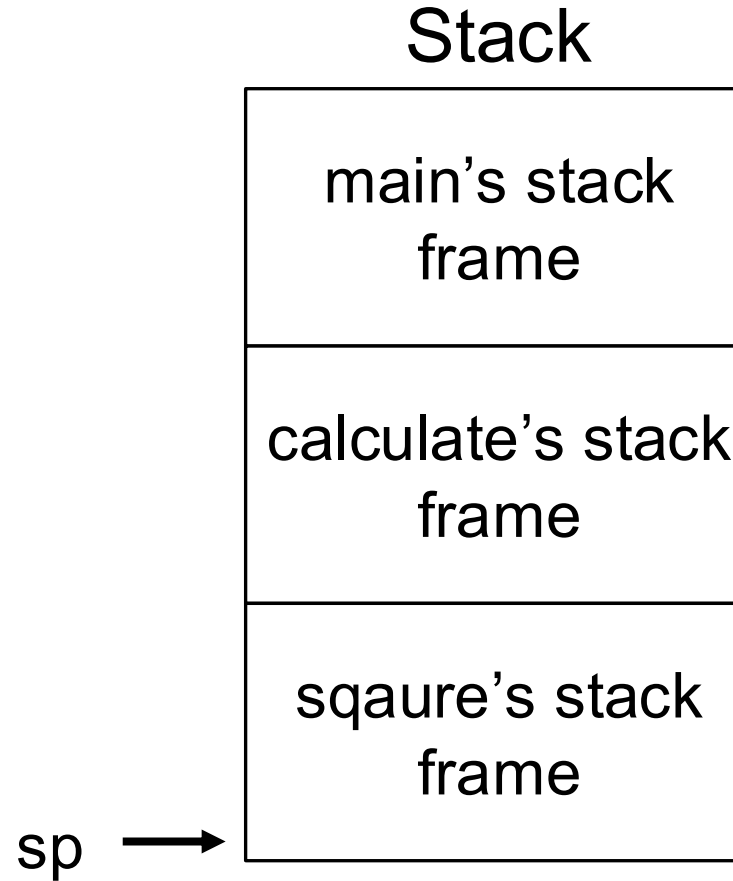
How you will draw this memory diagram on paper

```
int square(int x) {  
    int result = x * x;  
    return result;  
}  
  
int addOne(int x) {  
    int result = x + 1;  
    return result;  
}  
  
int calculate() {  
    int answer =  
        addOne(square(5));  
    return answer;  
}  
  
int main() {  
    calculate();  
}
```



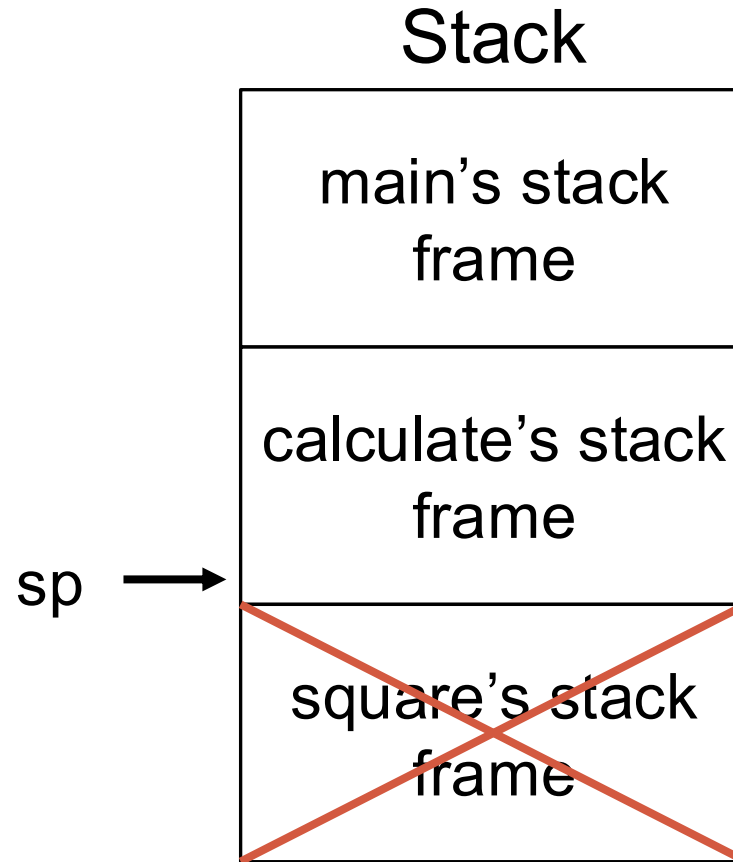
How you will draw this memory diagram on paper

```
int square(int x) {  
    int result = x * x;  
    return result;  
}  
  
int addOne(int x) {  
    int result = x + 1;  
    return result;  
}  
  
int calculate() {  
    int answer =  
        addOne(square(5));  
    return answer;  
}  
  
int main() {  
    calculate();  
}
```



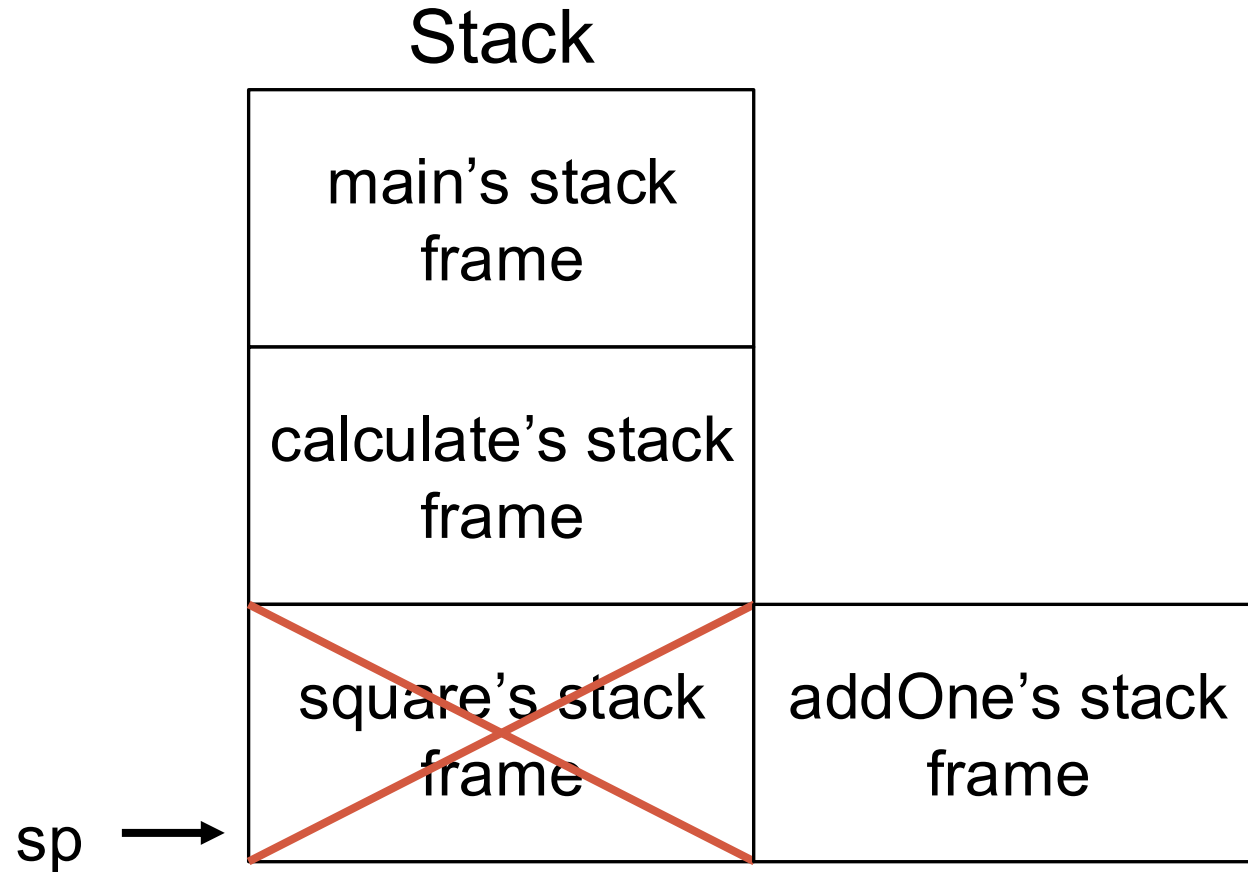
How you will draw this memory diagram on paper

```
int square(int x) {  
    int result = x * x;  
    return result;  
}  
  
int addOne(int x) {  
    int result = x + 1;  
    return result;  
}  
  
int calculate() {  
    int answer =  
        addOne(square(5));  
    return answer;  
}  
  
int main() {  
    calculate();  
}
```



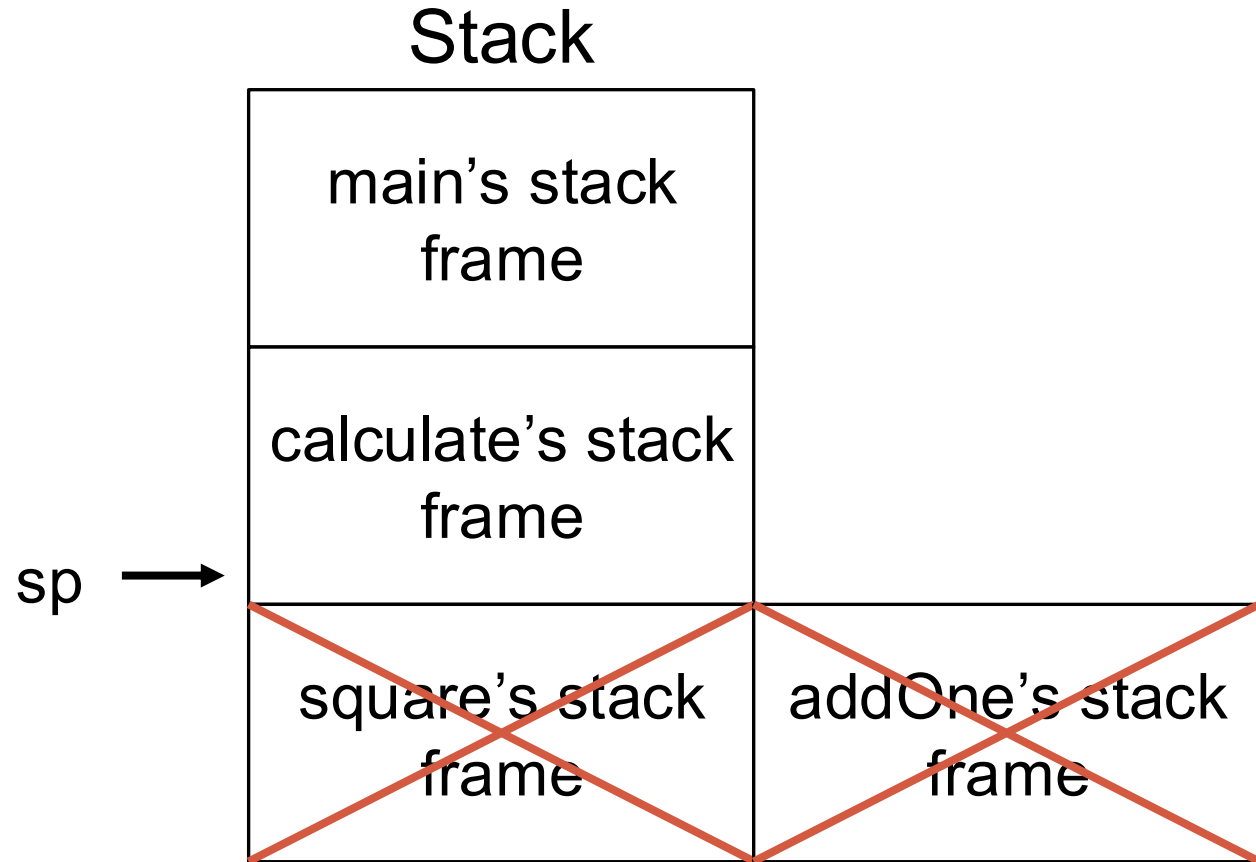
How you will draw this memory diagram on paper

```
int square(int x) {  
    int result = x * x;  
    return result;  
}  
  
int addOne(int x) {  
    int result = x + 1;  
    return result;  
}  
  
int calculate() {  
    int answer =  
        addOne(square(5));  
    return answer;  
}  
  
int main() {  
    calculate();  
}
```



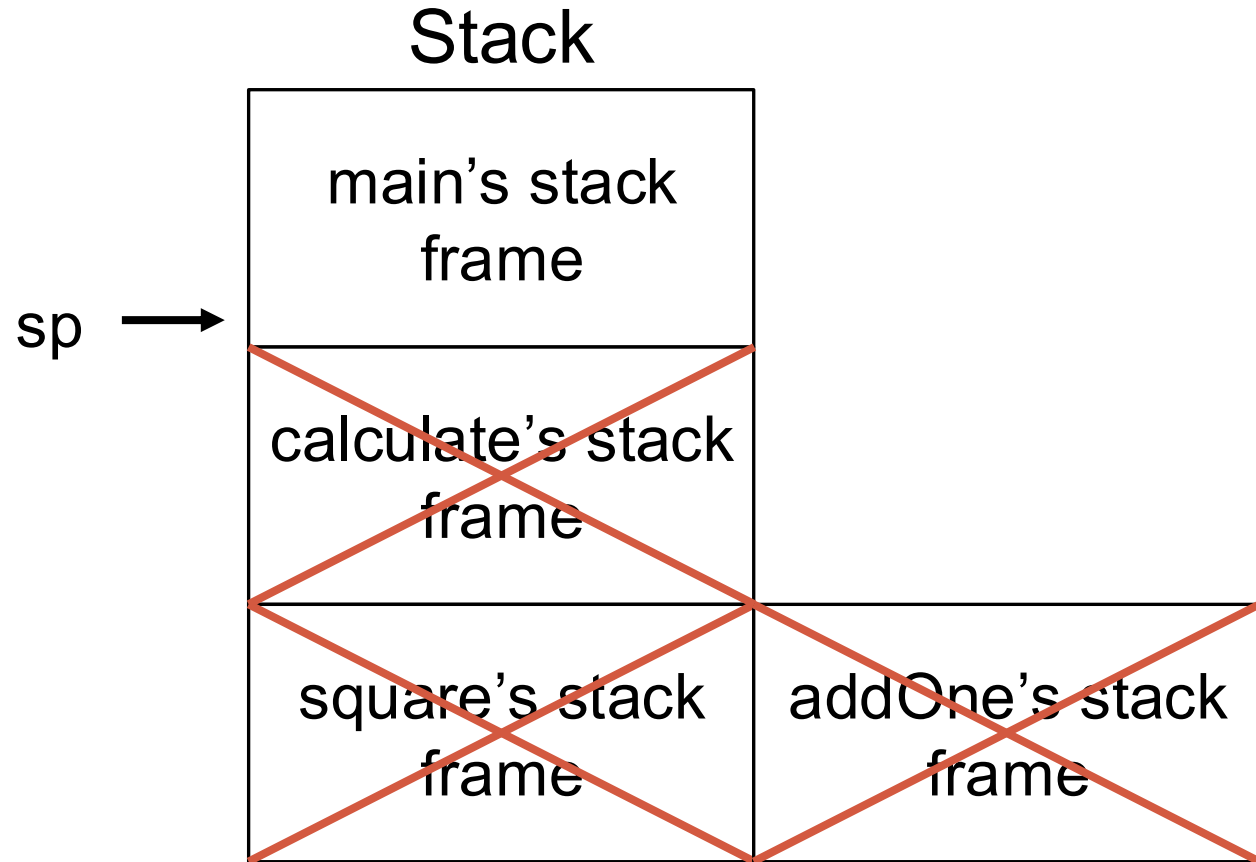
How you will draw this memory diagram on paper

```
int square(int x) {  
    int result = x * x;  
    return result;  
}  
  
int addOne(int x) {  
    int result = x + 1;  
    return result;  
}  
  
int calculate() {  
    int answer =  
        addOne(square(5));  
    return answer;  
}  
  
int main() {  
    calculate();  
}
```



How you will draw this memory diagram on paper

```
int square(int x) {  
    int result = x * x;  
    return result;  
}  
  
int addOne(int x) {  
    int result = x + 1;  
    return result;  
}  
  
int calculate() {  
    int answer =  
        addOne(square(5));  
    return answer;  
}  
  
int main() {  
    calculate();  
}
```

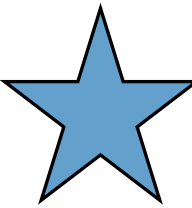


Example 3

Draw the memory diagram just before main returns.

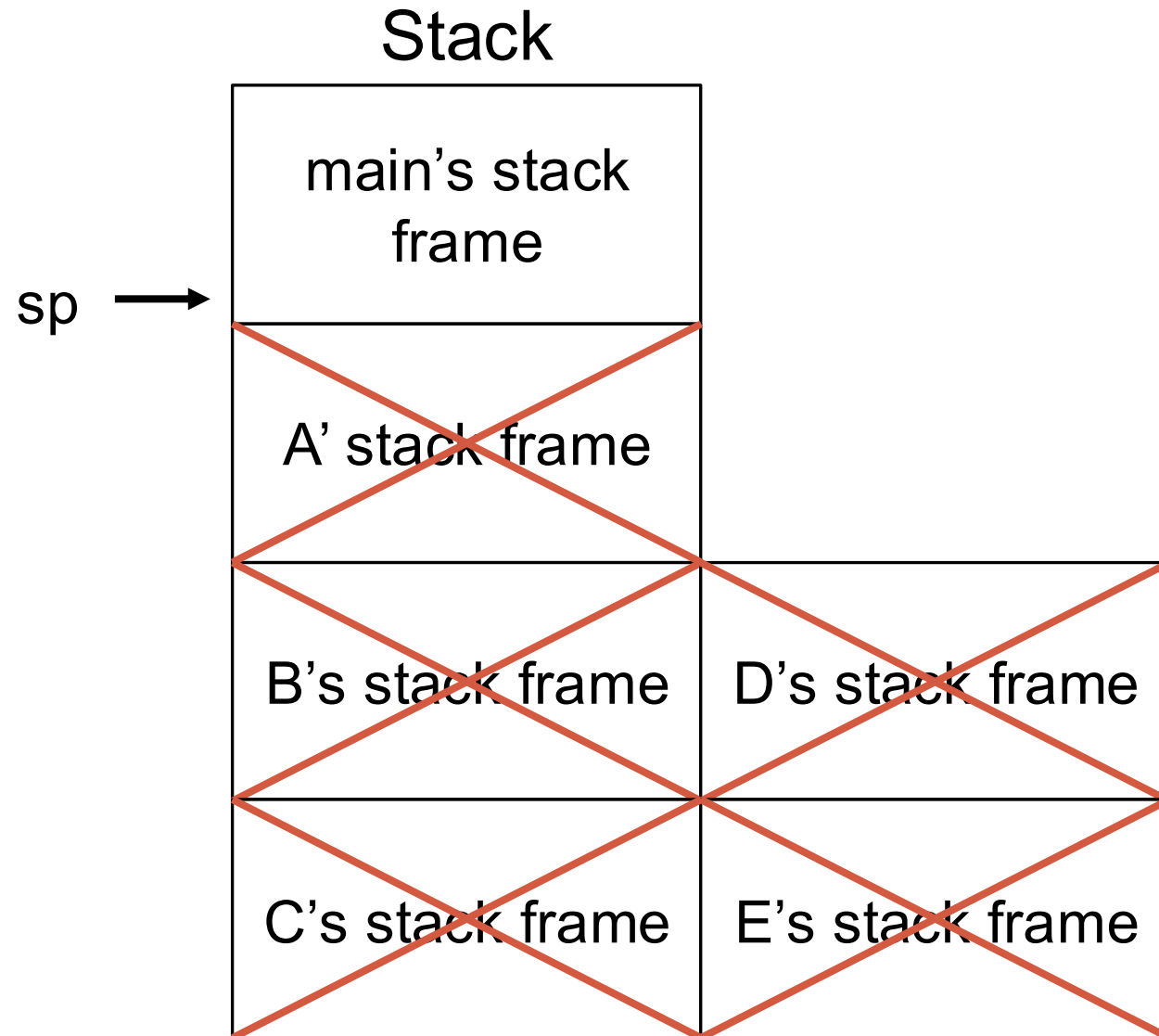
Stack

```
void E() {  
    int e = 50;  
}  
void D() {  
    int d = 40;  
    E();  
}  
void C() {  
    int c = 30;  
}  
void B() {  
    int b = 20;  
    C();  
}  
void A() {  
    int a = 10;  
    B();  
    D();  
}  
int main() {  
    A();  
}
```



Draw the memory diagram just before main returns.

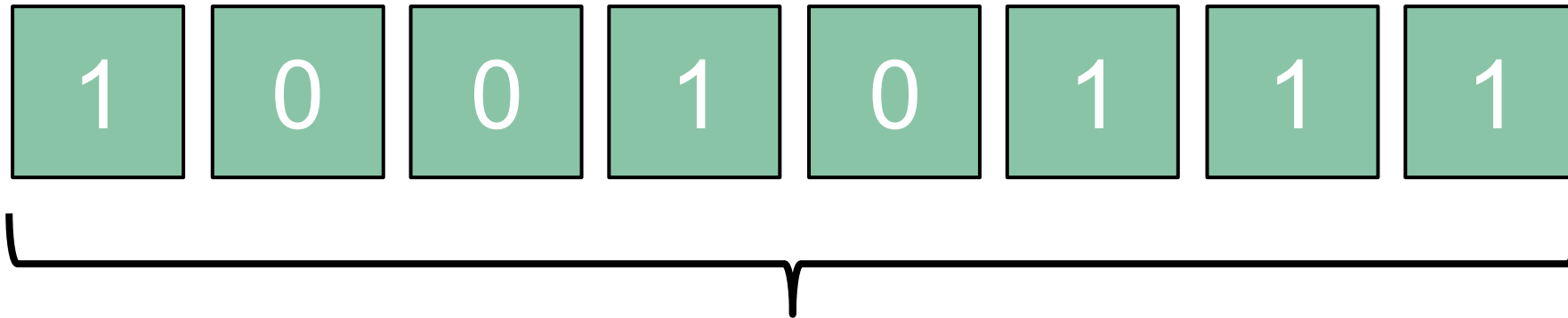
```
void E() {  
    int e = 50;  
}  
void D() {  
    int d = 40;  
    E();  
}  
void C() {  
    int c = 30;  
}  
void B() {  
    int b = 20;  
    C();  
}  
void A() {  
    int a = 10;  
    B();  
    D();  
}  
int main() {  
    A();  
}
```



High-Level Memory Diagrams

Byte

- A **bit** is the smallest piece of data
 - It can be 0 or 1
- A **byte** is 8 bits



8 bits = 1 byte

Fixed-Size Integers

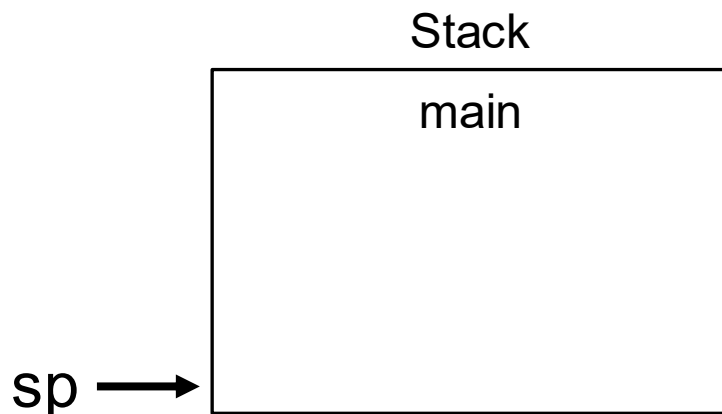
- `int` can be different sizes on different systems
- **Fixed-size integers** are integers with guaranteed sizes, giving you control over memory and value range.
- This makes code:
 - More predictable
 - More portable
 - Better for memory-limited systems (like embedded devices)
- `#include <stdint.h>`

Fixed-Size Integers

Type	Bits Used	Bytes	When to use
<code>int8_t</code>	8 bits	1 byte	tiny values, sensors, raw data
<code>int16_t</code>	16 bits	2 bytes	small counters, audio samples
<code>int32_t</code>	32 bits	4 bytes	general-purpose numbers
<code>int64_t</code>	64 bits	8 bytes	large counts, timestamps, big calculations

Memory Diagram Example

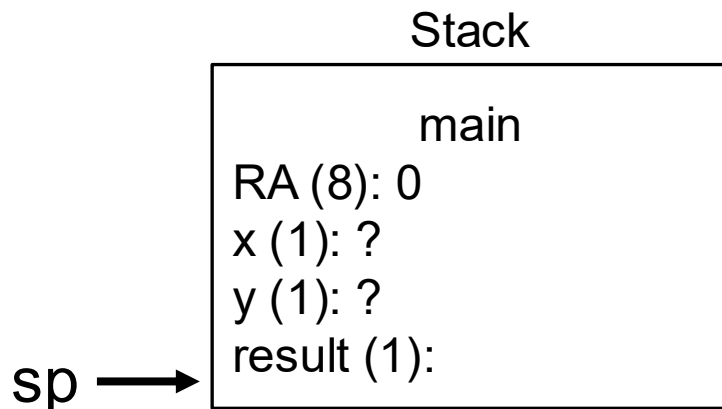
- When `main` starts:
 - The stack has one frame for `main` which contains space for its
 - **Return address**
 - **Arguments**
 - **Local Variables**



```
1  #include <stdint.h>
2
3  int8_t add(int8_t a, int8_t b);
4
5  int main(void) {
6      int8_t x = 5, y = -10;
7      int8_t result = 2 * add(x, y);
8      return 0;
9  }
10
11 int8_t add(int8_t a, int8_t b) {
12     int8_t result = a + b;
13     return result;
14 }
```

Memory Diagram Example

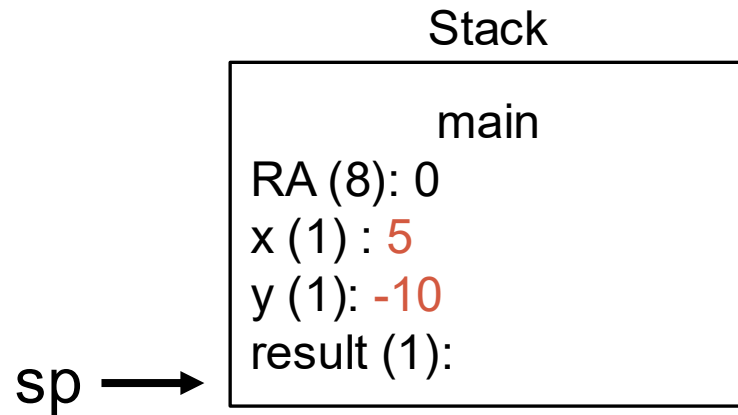
- When `main` starts:
 - The stack has one frame for `main` which contains space for its
 - **Return address:** `main` returns to the operating system (we'll call the return address here 0)
 - **Arguments:** There are no arguments
 - **Local Variables:** `x`, `y`, `result`



```
1  #include <stdint.h>
2
3  int8_t add(int8_t a, int8_t b);
4
5  int main(void) {
6      int8_t x = 5, y = -10;
7      int8_t result = 2 * add(x, y);
8      return 0;
9  }
10
11 int8_t add(int8_t a, int8_t b) {
12     int8_t result = a + b;
13     return result;
14 }
```

Memory Diagram Example

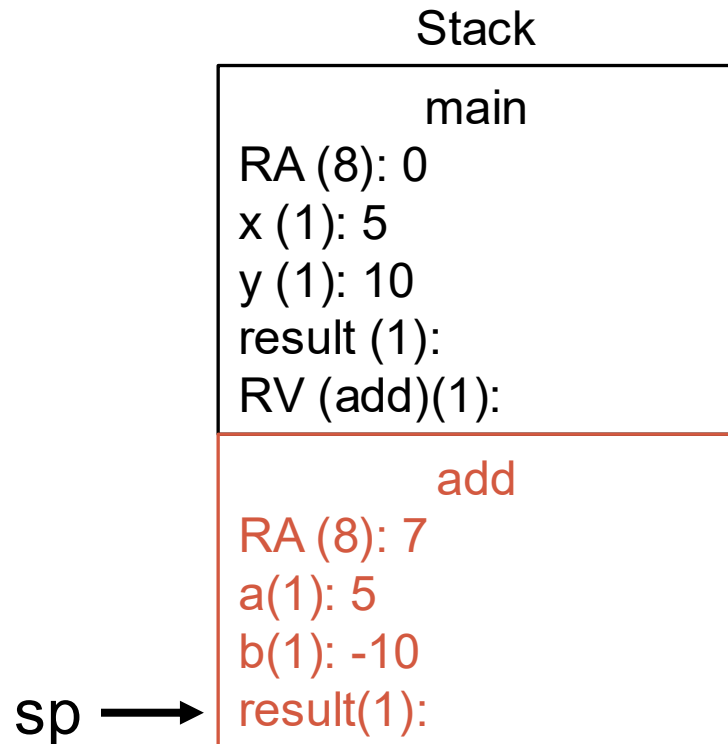
- When variables are initialized, their values are stored in their designated locations within the stack frame.



```
1  #include <stdint.h>
2
3  int8_t add(int8_t a, int8_t b);
4
5  int main(void) {
6      int8_t x = 5, y = -10;
7      int8_t result = 2 * add(x, y);
8      return 0;
9  }
10
11 int8_t add(int8_t a, int8_t b) {
12     int8_t result = a + b;
13     return result;
14 }
```

Memory Diagram Example

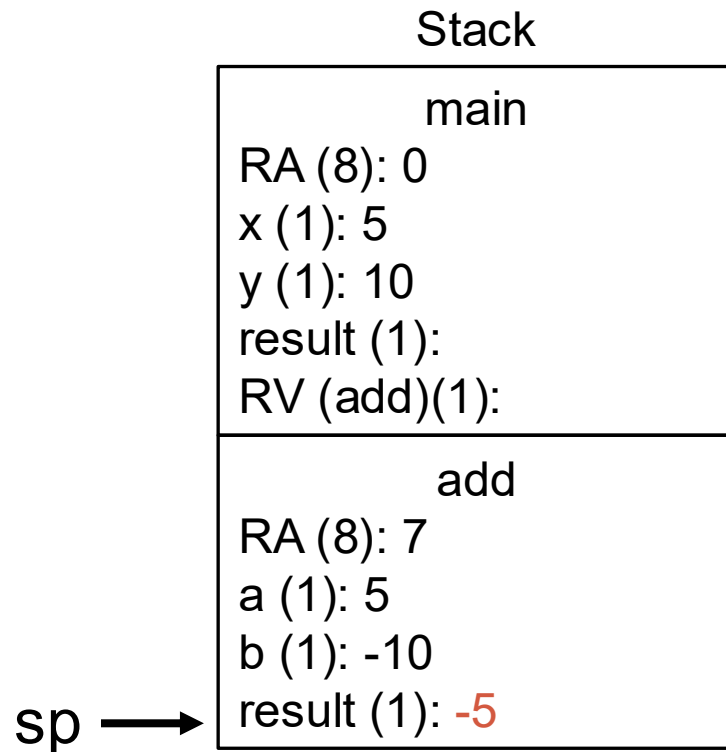
- Then a frame for `add` is created which contains space for its arguments, local variables, and return address (RA).
 - RA is the line the program will return to



```
1  #include <stdint.h>
2
3  int8_t add(int8_t a, int8_t b);
4
5  int main(void) {
6      int8_t x = 5, y = -10;
7      int8_t result = 2 * add(x, y);
8      return 0;
9  }
10
11 int8_t add(int8_t a, int8_t b) {
12     int8_t result = a + b;
13     return result;
14 }
```

Memory Diagram Example

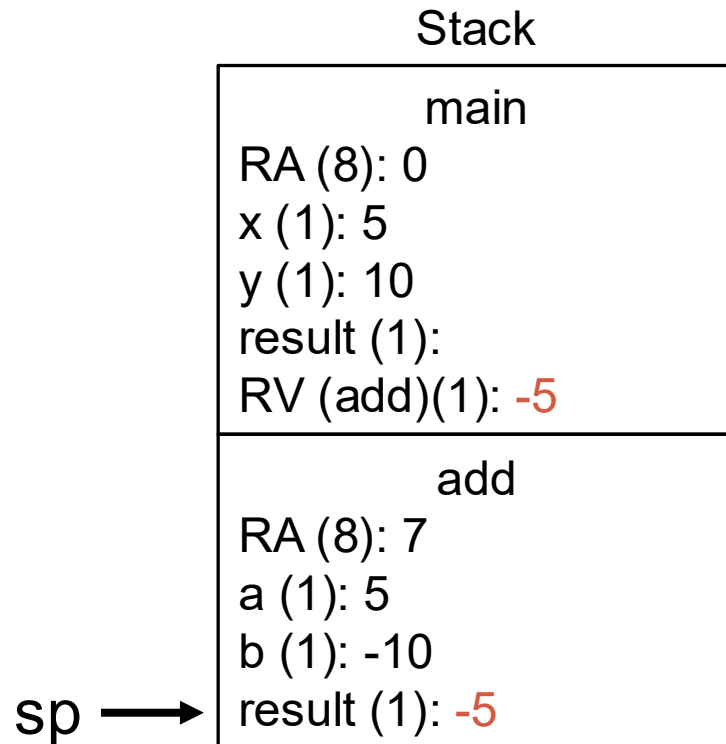
- When a value is assigned to `result`, it is stored in its allocated spot within the stack frame.



```
1  #include <stdint.h>
2
3  int8_t add(int8_t a, int8_t b);
4
5  int main(void) {
6      int8_t x = 5, y = -10;
7      int8_t result = 2 * add(x, y);
8      return 0;
9  }
10
11 int8_t add(int8_t a, int8_t b) {
12     int8_t result = a + b;
13     return result;
14 }
```

Memory Diagram Example

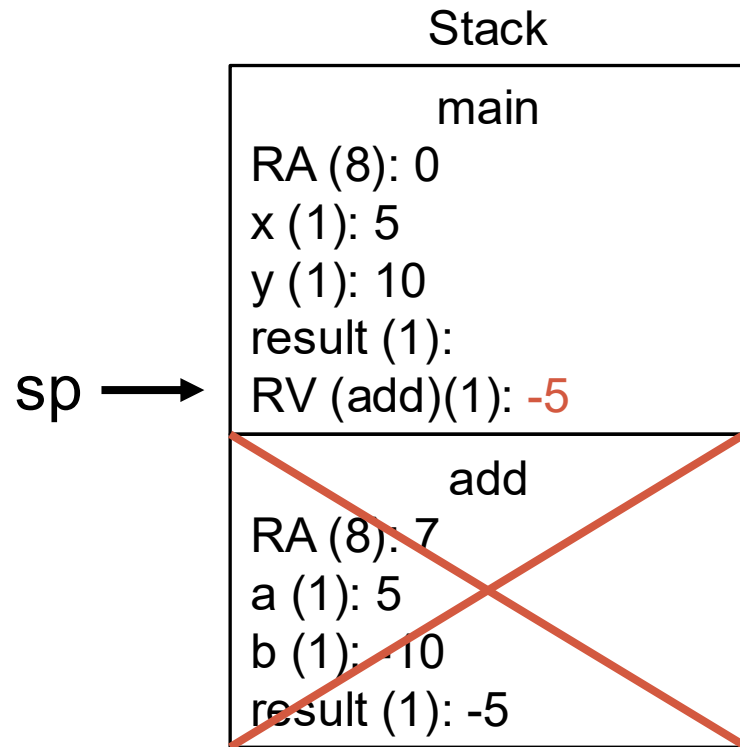
- Before `add` returns, it will place the return value in `main`'s frame so that `main` can access it.



```
1  #include <stdint.h>
2
3  int8_t add(int8_t a, int8_t b);
4
5  int main(void) {
6      int8_t x = 5, y = -10;
7      int8_t result = 2 * add(x, y);
8      return 0;
9  }
10
11 int8_t add(int8_t a, int8_t b) {
12     int8_t result = a + b;
13     return result;
14 }
```

Memory Diagram Example

- When `add` returns, its frame is deallocated.

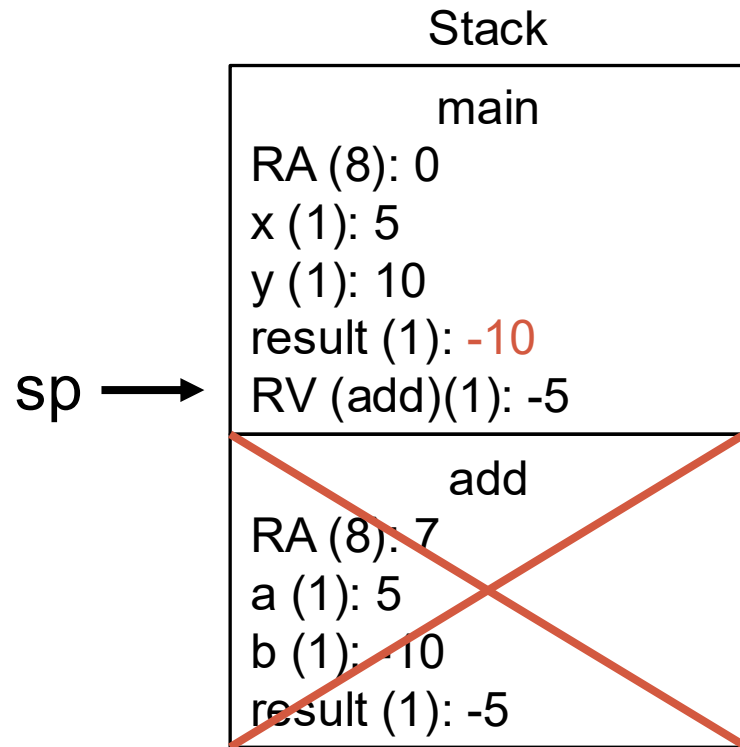


```
1  #include <stdint.h>
2
3  int8_t add(int8_t a, int8_t b);
4
5  int main(void) {
6      int8_t x = 5, y = -10;
7      int8_t result = 2 * add(x, y);
8      return 0;
9  }
10
11 int8_t add(int8_t a, int8_t b) {
12     int8_t result = a + b;
13     return result;
14 }
```

To represent that a frame has been deallocated, place an X over it.

Memory Diagram Example

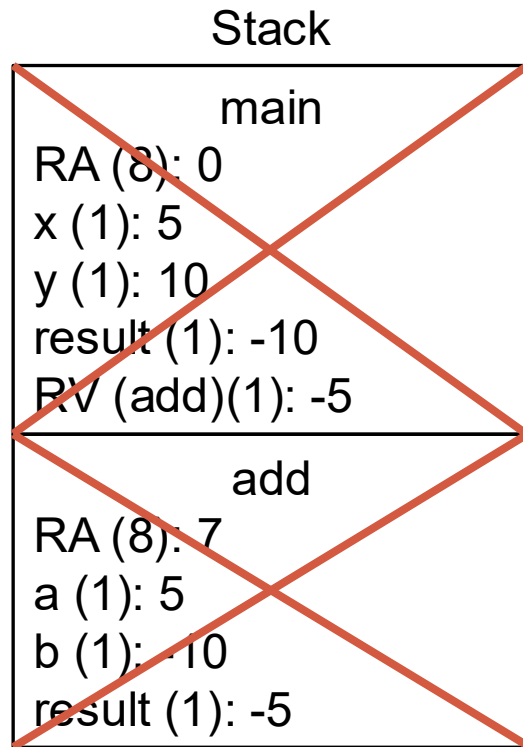
- The local variable `result` is assigned



```
1  #include <stdint.h>
2
3  int8_t add(int8_t a, int8_t b);
4
5  int main(void) {
6      int8_t x = 5, y = -10;
7      int8_t result = 2 * add(x, y);
8      return 0;
9  }
10
11 int8_t add(int8_t a, int8_t b) {
12     int8_t result = a + b;
13     return result;
14 }
```

Memory Diagram Example

- When `main` returns, its frame is deallocated, the process terminates, and control returns to the operating system.



```
1  #include <stdint.h>
2
3  int8_t add(int8_t a, int8_t b);
4
5  int main(void) {
6      int8_t x = 5, y = -10;
7      int8_t result = 2 * add(x, y);
8      return 0;
9  }
10
11 int8_t add(int8_t a, int8_t b) {
12     int8_t result = a + b;
13     return result;
14 }
```

To represent that a frame has been deallocated, place an X over it.

Process

- A **process** is a running instance of a program.
- A program is passive
 - It is just code stored in memory
- A process is active
 - It represents that program in execution, with its own memory, CPU state, and resources.

Memory Diagram Example with an Array

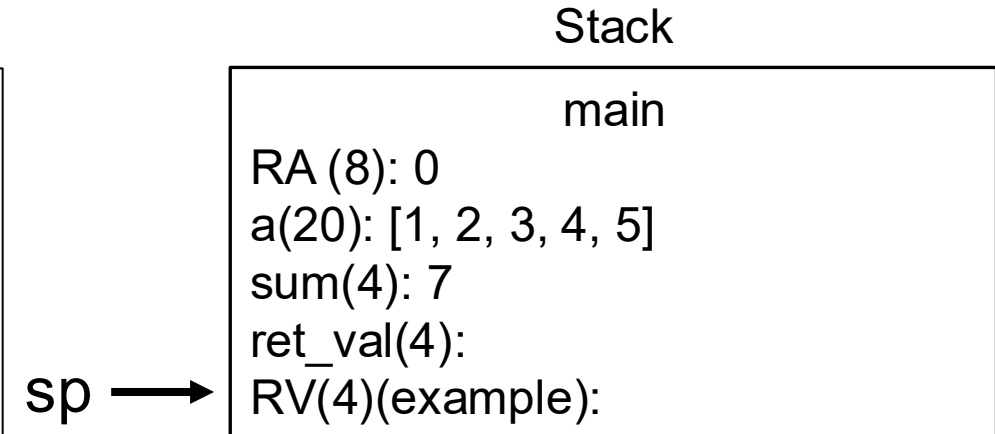
```
1  #include <stdint.h>
2
3  #define ARR_SIZE 5 ← Preprocessor Directive
4
5  int32_t example(int32_t arr[ARR_SIZE], int32_t size, int32_t sum) {
6      sum = 0;
7      for (int16_t i = 0; i < size; i++) {
8          sum += arr[i];
9          arr[i]++;
10     }
11     return arr[0];
12 }
13
14 int main(void) {
15     int32_t a[ARR_SIZE] = {1, 2, 3, 4, 5};
16     int32_t sum = 5;
17     int32_t ret_val = example(a, ARR_SIZE, sum);
18 }
```

Preprocessor Directive

- When the preprocessor sees a `#define`, it replaces every occurrence of a name with its defined value literally, as if you had typed it yourself.
- What this means for memory diagrams:
 - The preprocessor directive is not stored in the stack, it ends up as part of the code.

Memory Diagram Example with an Array

```
1  #include <stdint.h>
2
3  #define ARR_SIZE 5
4
5  int32_t example(int32_t arr[ARR_SIZE],
6                 int32_t size, int32_t sum) {
7      sum = 0;
8      for (int16_t i = 0; i < size; i++) {
9          sum += arr[i];
10         arr[i]++;
11     }
12     return arr[0];
13 }
14
15 int main(void) {
16     int32_t a[ARR_SIZE] = {1, 2, 3, 4, 5};
17     int32_t sum = 7;
18     int32_t ret_val = example(a, ARR_SIZE, sum);
19 }
```



Arrays are not copied
when passed to
functions. Only a pointer
to the first element is
passed.

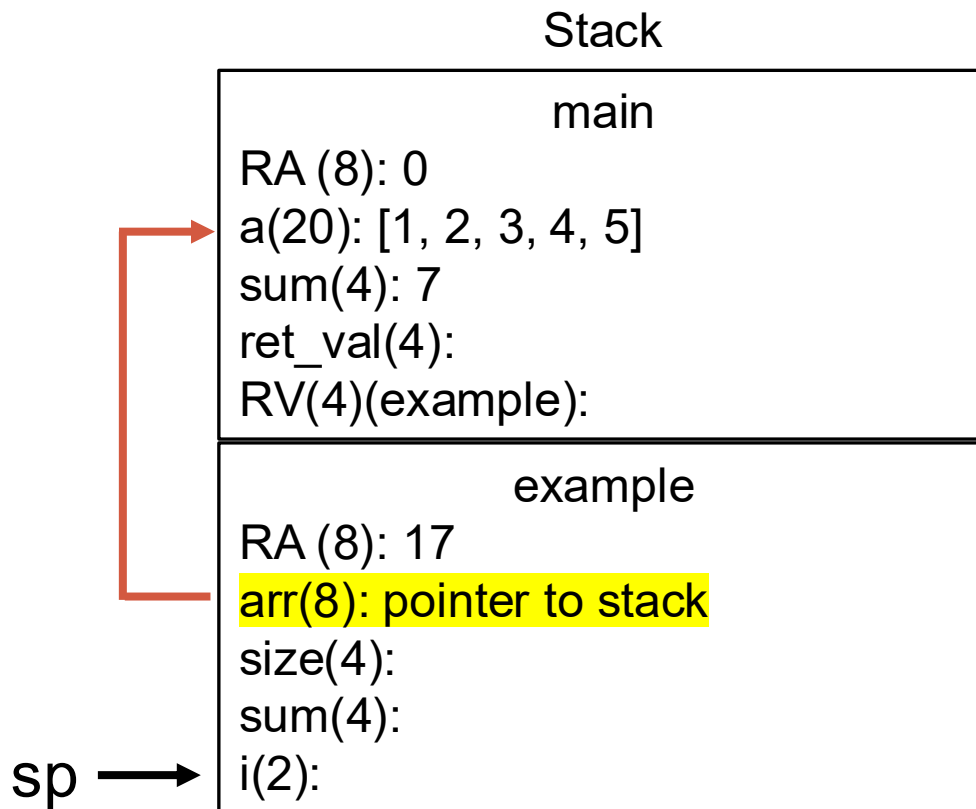
All pointers are 8 bytes.

Pass by Pointer

- When an array is passed to a function, the function receives a pointer to the original array.
- What does this mean for our program?
 - The array is not copied
 - Both `main's a` and `example's arr` refer to the same array in memory
 - Changes made in `example` affect the original array

Memory Diagram Example with an Array

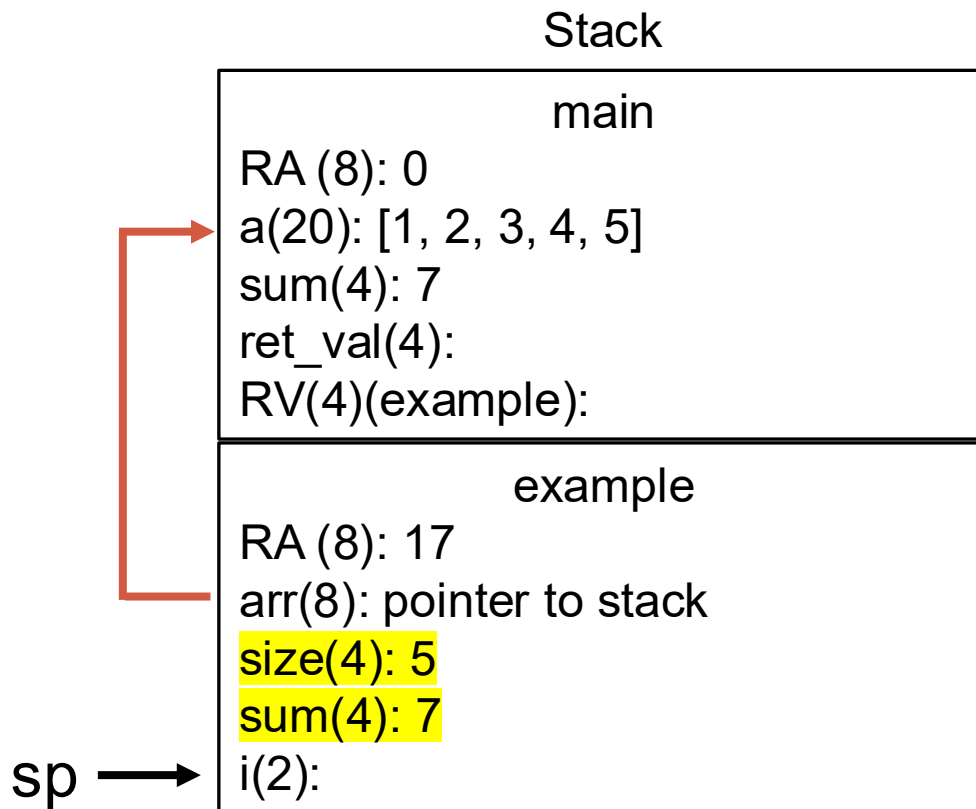
- To represent a pointer to a variable stored on the stack
 - Write “pointer to stack”
 - Draw an arrow to the variable



```
1  #include <stdint.h>
2
3  #define ARR_SIZE 5
4
5  int32_t example(int32_t arr[ARR_SIZE],
6                 int32_t size, int32_t sum) {
7      sum = 0;
8      for (int16_t i = 0; i < size; i++) {
9          sum += arr[i];
10         arr[i]++;
11     }
12     return arr[0];
13 }
14
15 int main(void) {
16     int32_t a[ARR_SIZE] = {1, 2, 3, 4, 5};
17     int32_t sum = 7;
18     int32_t ret_val = example(a, ARR_SIZE, sum);
19 }
```

Memory Diagram Example with an Array

- `size` and `sum` are passed by value



```
1  #include <stdint.h>
2
3  #define ARR_SIZE 5
4
5  int32_t example(int32_t arr[ARR_SIZE],
6                  int32_t size, int32_t sum) {
7      sum = 0;
8      for (int16_t i = 0; i < size; i++) {
9          sum += arr[i];
10         arr[i]++;
11     }
12     return arr[0];
13 }
14
15 int main(void) {
16     int32_t a[ARR_SIZE] = {1, 2, 3, 4, 5};
17     int32_t sum = 7;
18     int32_t ret_val = example(a, ARR_SIZE, sum);
19 }
```

Pass-by-value

A copy of the variable is passed

The function works with its own copy

Changes do NOT affect the original variable

Pass-by-pointer

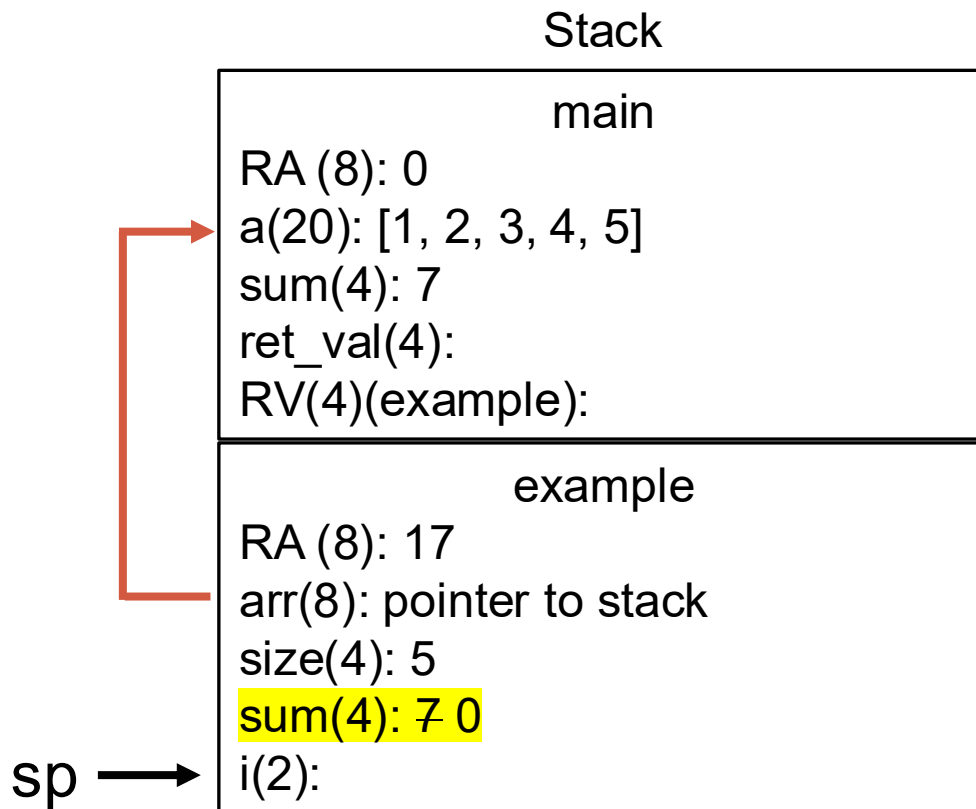
A pointer (address) to the variable is passed

The function accesses the original data

Changes DO affect the original variable

Memory Diagram Example with an Array

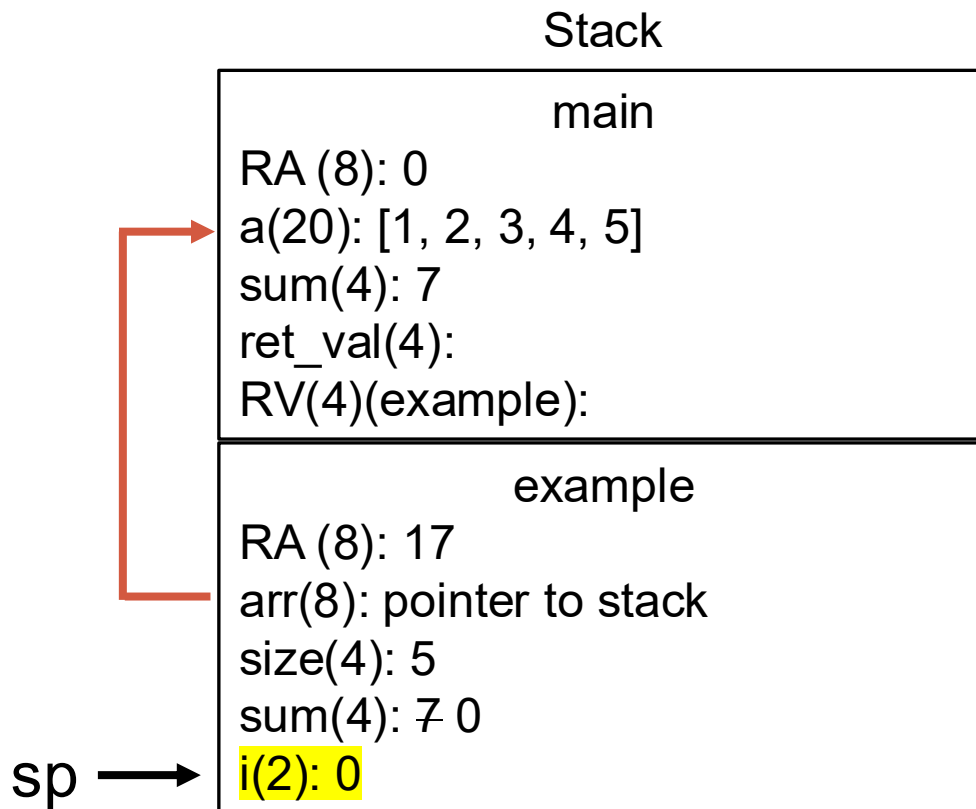
- `sum` becomes 0.
- Notice that `sum` in `main`'s frame does not change because `sum` was passed by value
- Be sure your memory diagram shows old values by crossing them out.



```
1  #include <stdint.h>
2
3  #define ARR_SIZE 5
4
5  int32_t example(int32_t arr[ARR_SIZE],
6                 int32_t size, int32_t sum) {
7      sum = 0;
8      for (int16_t i = 0; i < size; i++) {
9          sum += arr[i];
10         arr[i]++;
11     }
12     return arr[0];
13 }
14
15 int main(void) {
16     int32_t a[ARR_SIZE] = {1, 2, 3, 4, 5};
17     int32_t sum = 7;
18     int32_t ret_val = example(a, ARR_SIZE, sum);
19 }
```

Memory Diagram Example with an Array

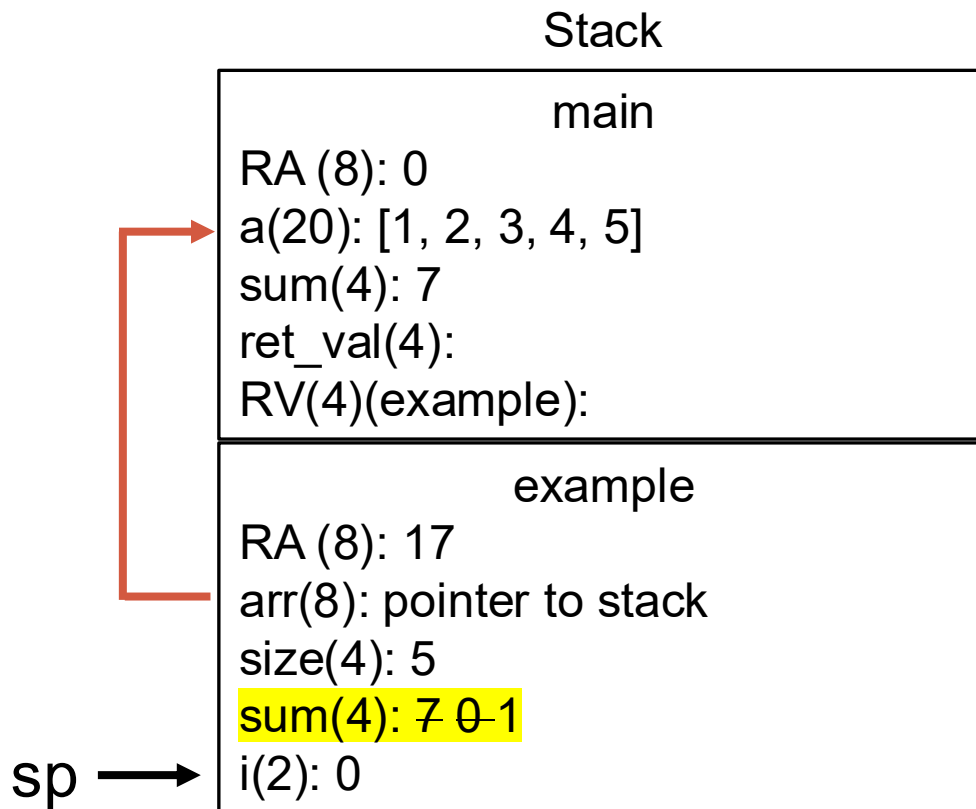
- `i` is initialized to 0



```
1  #include <stdint.h>
2
3  #define ARR_SIZE 5
4
5  int32_t example(int32_t arr[ARR_SIZE],
6                  int32_t size, int32_t sum) {
7      sum = 0;
8      for (int16_t i = 0; i < size; i++) {
9          sum += arr[i];
10         arr[i]++;
11     }
12     return arr[0];
13 }
14
15 int main(void) {
16     int32_t a[ARR_SIZE] = {1, 2, 3, 4, 5};
17     int32_t sum = 7;
18     int32_t ret_val = example(a, ARR_SIZE, sum);
19 }
```

Memory Diagram Example with an Array

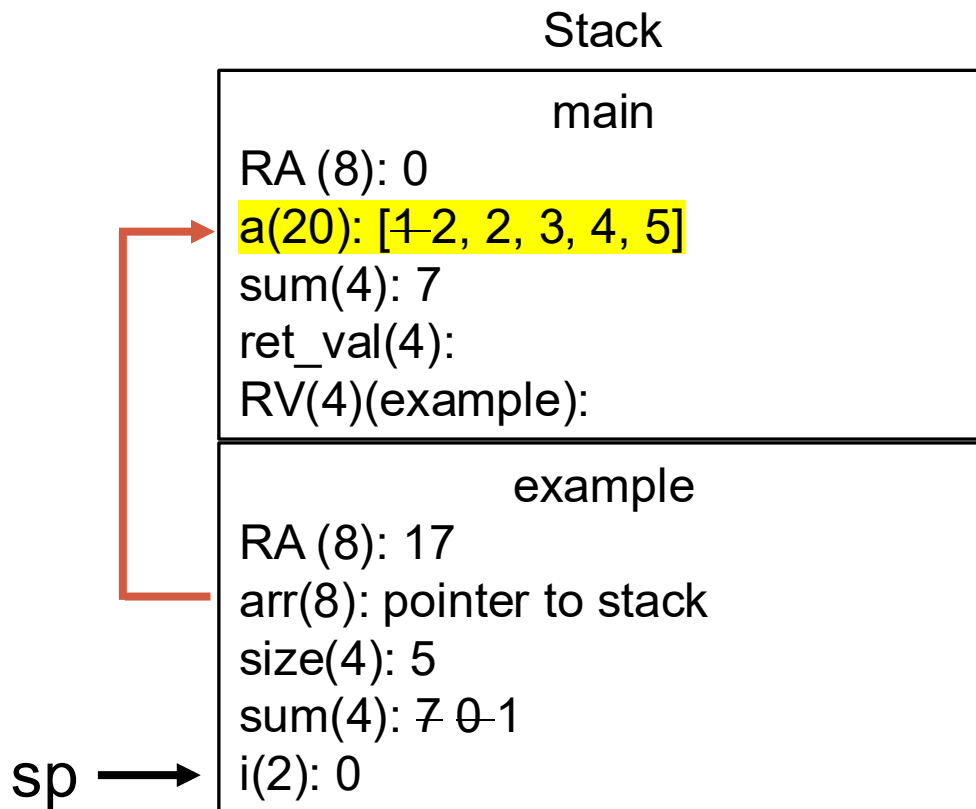
- `sum` is updated
- Again, notice that only `example`'s copy is updated.



```
1  #include <stdint.h>
2
3  #define ARR_SIZE 5
4
5  int32_t example(int32_t arr[ARR_SIZE],
6                 int32_t size, int32_t sum) {
7      sum = 0;
8      for (int16_t i = 0; i < size; i++) {
9          sum += arr[i];
10         arr[i]++;
11     }
12     return arr[0];
13 }
14
15 int main(void) {
16     int32_t a[ARR_SIZE] = {1, 2, 3, 4, 5};
17     int32_t sum = 7;
18     int32_t ret_val = example(a, ARR_SIZE, sum);
19 }
```

Memory Diagram Example with an Array

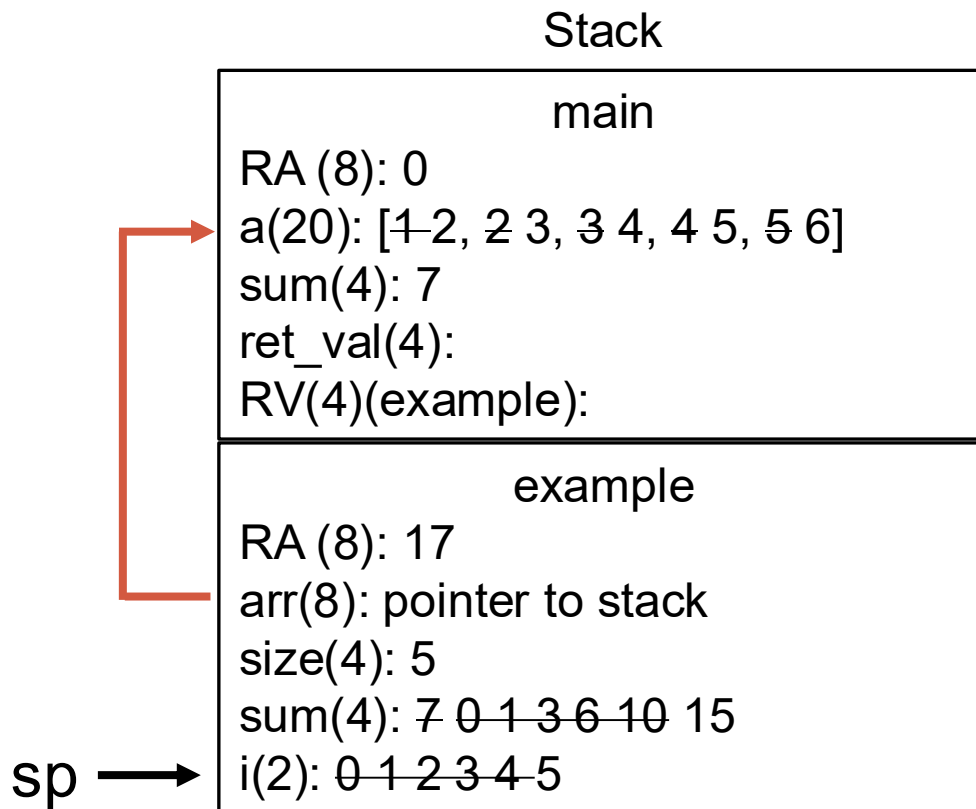
- `arr[0]` is updated
- Notice, this updates the array in `main`'s frame!



```
1  #include <stdint.h>
2
3  #define ARR_SIZE 5
4
5  int32_t example(int32_t arr[ARR_SIZE],
6                 int32_t size, int32_t sum) {
7      sum = 0;
8      for (int16_t i = 0; i < size; i++) {
9          sum += arr[i];
10         arr[i]++;
11     }
12     return arr[0];
13 }
14
15 int main(void) {
16     int32_t a[ARR_SIZE] = {1, 2, 3, 4, 5};
17     int32_t sum = 7;
18     int32_t ret_val = example(a, ARR_SIZE, sum);
19 }
```

Memory Diagram Example with an Array

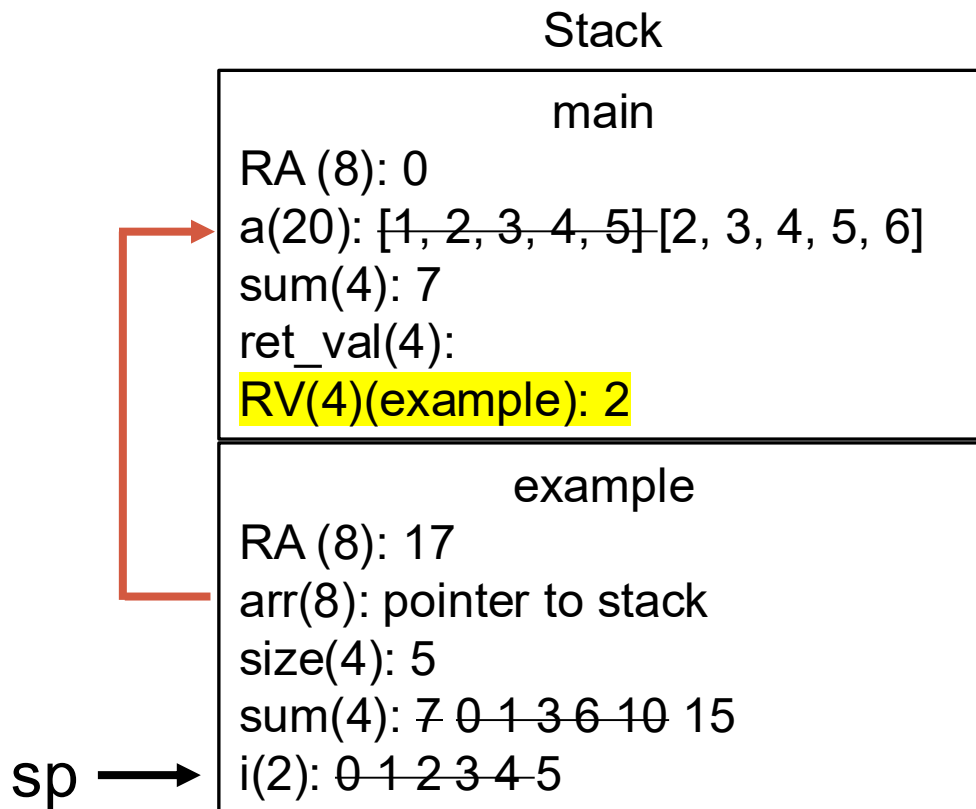
- Complete the loop.
- Notice that `i` ends at 5



```
1  #include <stdint.h>
2
3  #define ARR_SIZE 5
4
5  int32_t example(int32_t arr[ARR_SIZE],
6                  int32_t size, int32_t sum) {
7      sum = 0;
8      for (int16_t i = 0; i < size; i++) {
9          sum += arr[i];
10         arr[i]++;
11     }
12     return arr[0];
13 }
14
15 int main(void) {
16     int32_t a[ARR_SIZE] = {1, 2, 3, 4, 5};
17     int32_t sum = 7;
18     int32_t ret_val = example(a, ARR_SIZE, sum);
19 }
```

Memory Diagram Example with an Array

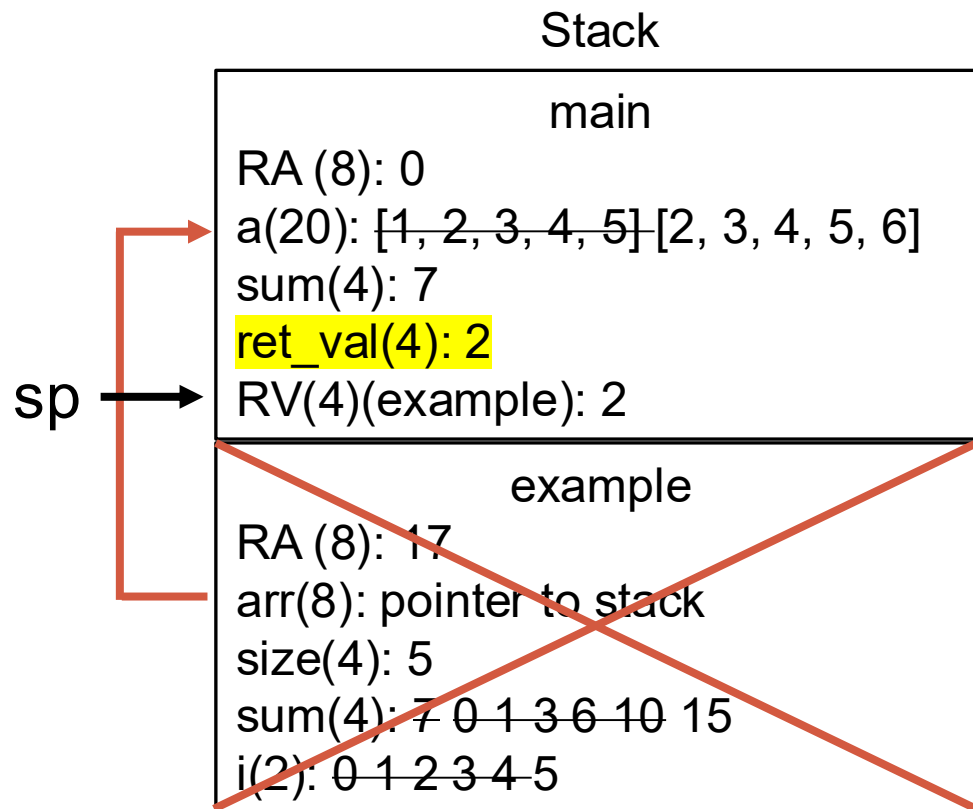
- Just before example returns, it places the return value in `main`'s frame.



```
1  #include <stdint.h>
2
3  #define ARR_SIZE 5
4
5  int32_t example(int32_t arr[ARR_SIZE],
6                 int32_t size, int32_t sum) {
7      sum = 0;
8      for (int16_t i = 0; i < size; i++) {
9          sum += arr[i];
10         arr[i]++;
11     }
12     return arr[0];
13 }
14
15 int main(void) {
16     int32_t a[ARR_SIZE] = {1, 2, 3, 4, 5};
17     int32_t sum = 7;
18     int32_t ret_val = example(a, ARR_SIZE, sum);
19 }
```

Memory Diagram Example with an Array

- When `example` returns, the return value is assigned to a local variable.



```
1  #include <stdint.h>
2
3  #define ARR_SIZE 5
4
5  int32_t example(int32_t arr[ARR_SIZE],
6                  int32_t size, int32_t sum) {
7      sum = 0;
8      for (int16_t i = 0; i < size; i++) {
9          sum += arr[i];
10         arr[i]++;
11     }
12     return arr[0];
13 }
14
15 int main(void) {
16     int32_t a[ARR_SIZE] = {1, 2, 3, 4, 5};
17     int32_t sum = 7;
18     int32_t ret_val = example(a, ARR_SIZE, sum);
19 }
```

Pass-by-value

A copy of the variable is passed

The function works with its own copy

Changes do NOT affect the original variable

Pass-by-pointer

A pointer (address) to the variable is passed

The function accesses the original data

Changes DO affect the original variable

Draw the Memory Diagram Just After example Returns

Stack

```
1  #include <stdint.h>
2  #include <stdio.h>
3
4  #define ARR_SIZE 3
5
6  int32_t example(int32_t a[ARR_SIZE], int32_t
    b[ARR_SIZE], int32_t size, int32_t c) {
7      for (int64_t i = 0; i < size; i++) {
8          a[i] = b[i] + i;
9          b[i] = a[i] - c;
10         c++;
11     }
12     return c;
13 }
14
15 int main(void) {
16     int32_t x[ARR_SIZE] = {10, 20, 30};
17     int32_t y[ARR_SIZE] = {40, 50, 60};
18     int32_t demo = x[0] + y[0];
19     int32_t ret_val = example(x, y, ARR_SIZE, demo);
20 }
```

Draw the Memory Diagram Just After example Returns



```
1 #include <stdint.h>
2 #include <stdio.h>
3
4 #define ARR_SIZE 3
5
6 int32_t example(int32_t a[ARR_SIZE], int32_t
7               b[ARR_SIZE], int32_t size, int32_t c) {
8     for (int64_t i = 0; i < size; i++) {
9         a[i] = b[i] + i;
10        b[i] = a[i] - c;
11        c++;
12    }
13    return c;
14 }
15
16 int main(void) {
17     int32_t x[ARR_SIZE] = {10, 20, 30};
18     int32_t y[ARR_SIZE] = {40, 50, 60};
19     int32_t demo = x[0] + y[0];
20     int32_t ret_val = example(x, y, ARR_SIZE, demo);
21 }
```

